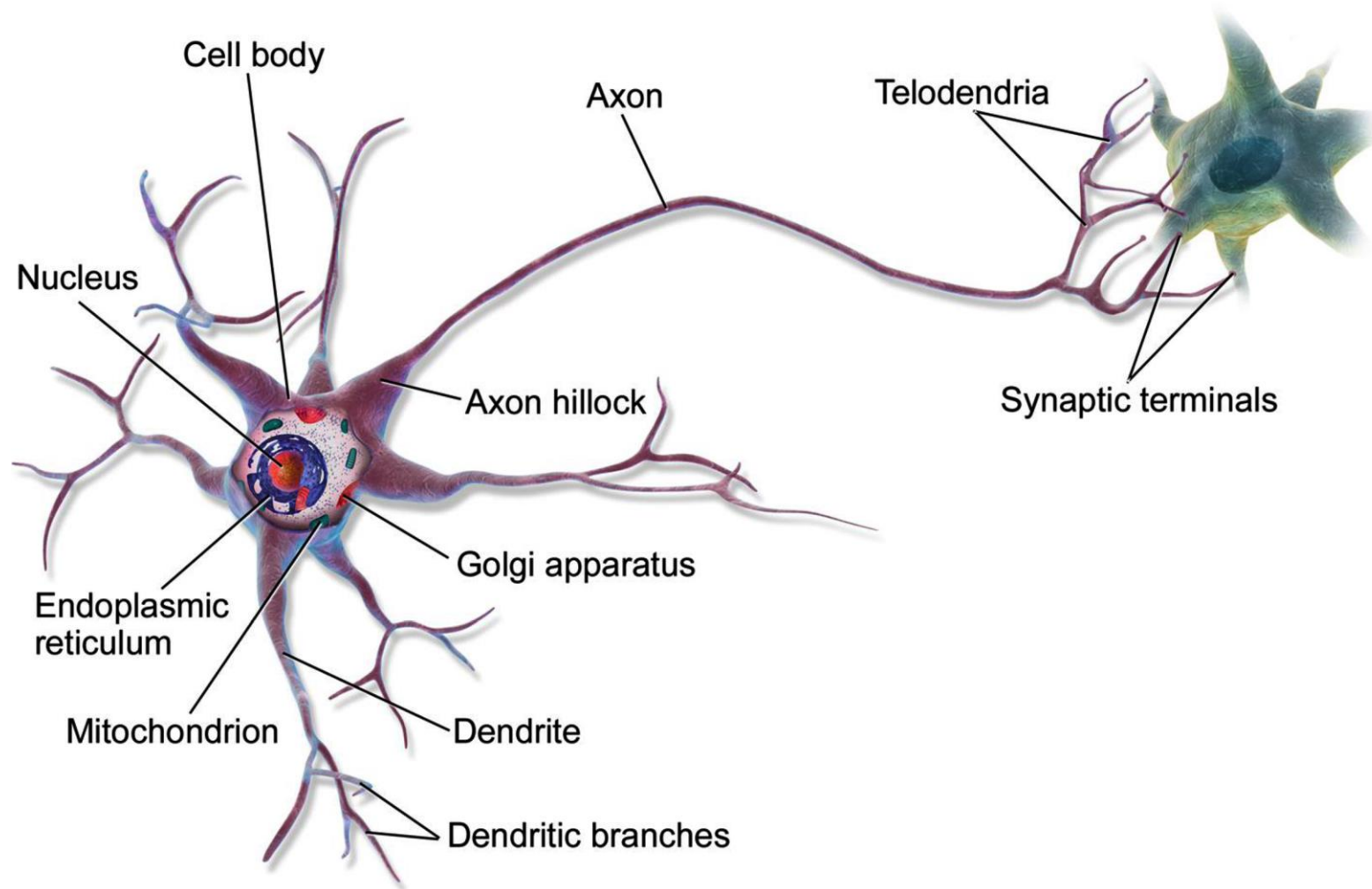


Topic 12

Neural Networks

A biological Neuron

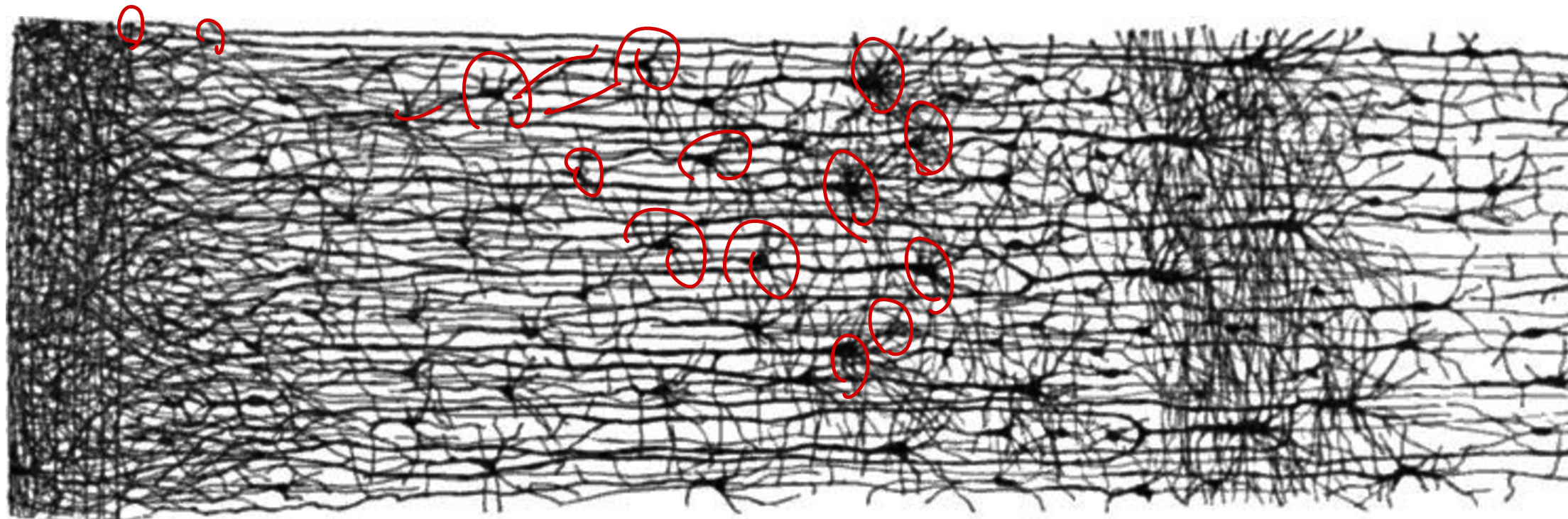


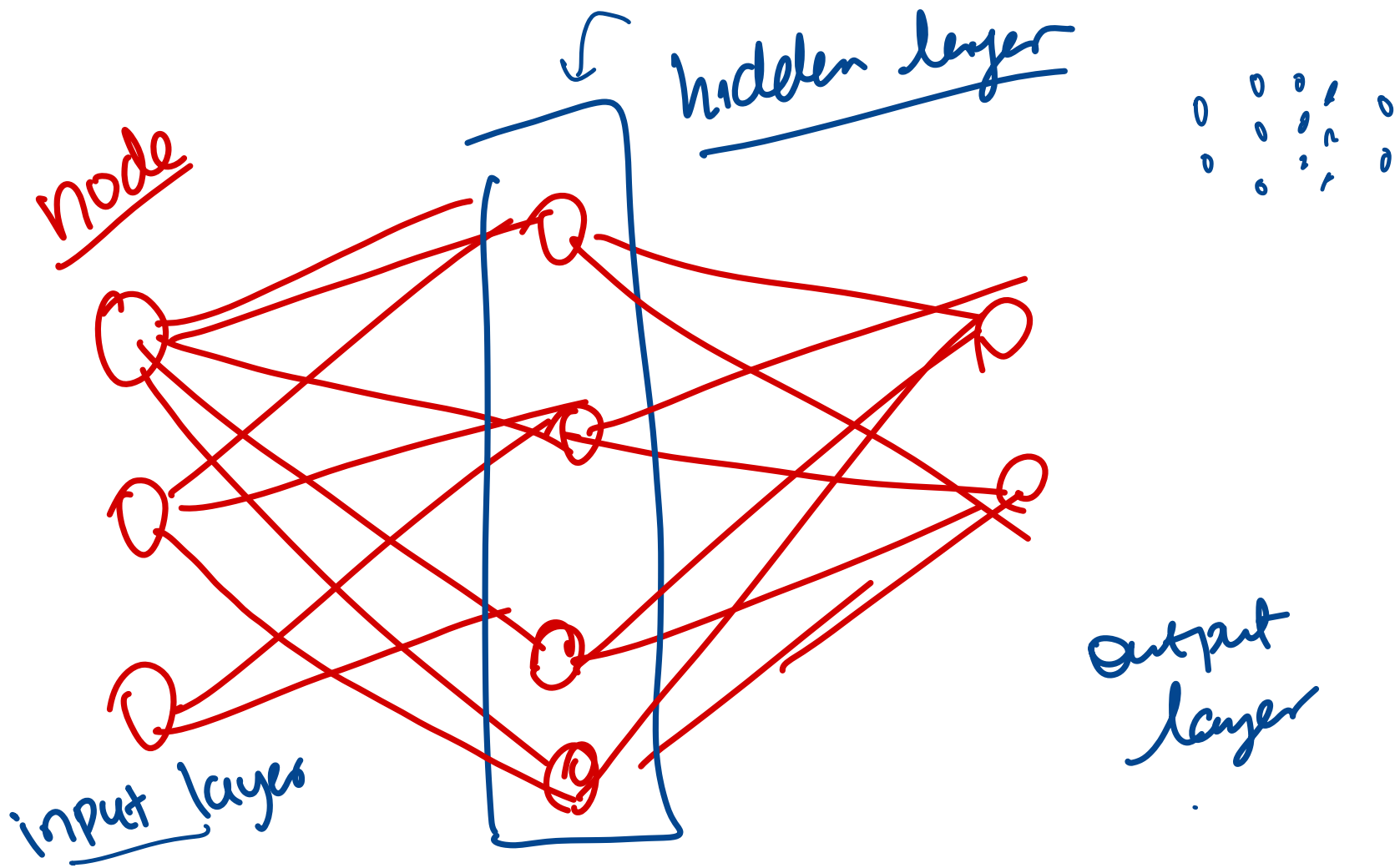
input

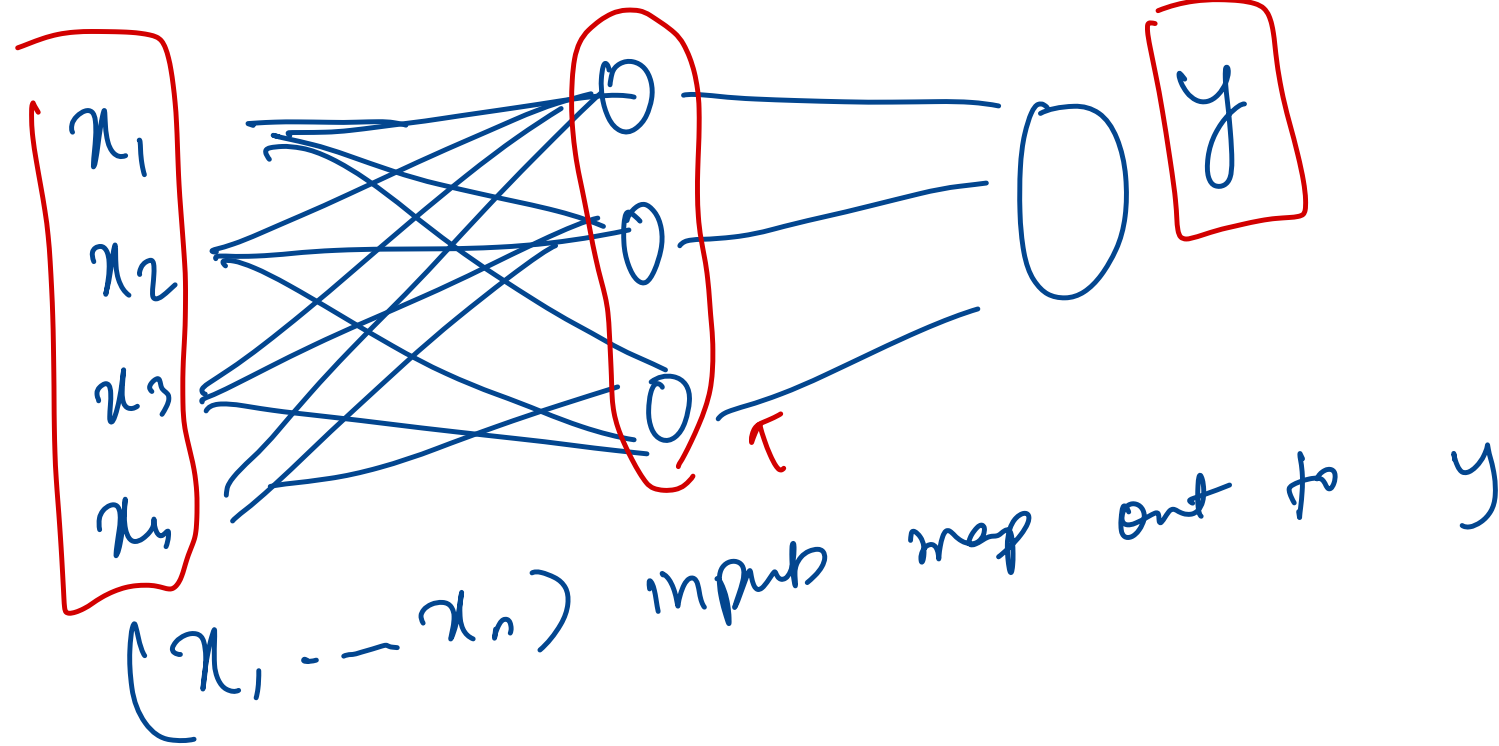
Layers in a biological Neural Network

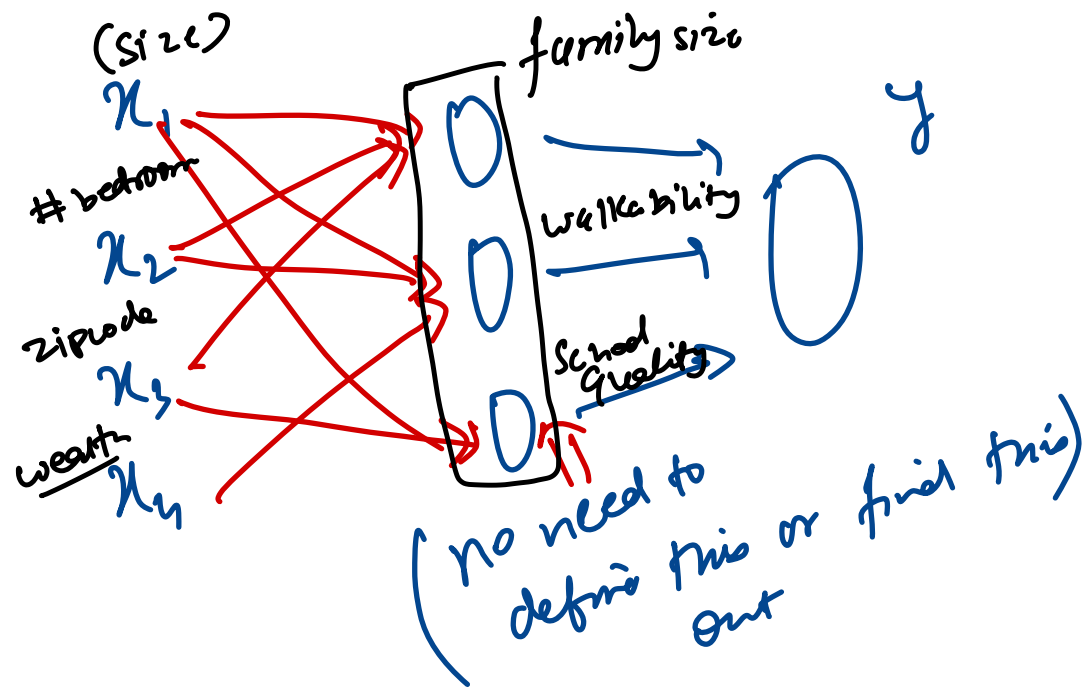
output

hidden









$(x_1 \dots x_n) \rightarrow$
map out to y

Supervised Learning

← ARTIFICIAL NN

Input(x) ↙	Output (y) ↙	Application
Home features	Price	Real Estate
Ad, user info ↙	Click on ad? (0/1)	Online Advertising
Image	Object (1,...,1000)	Photo tagging
Audio	Text transcript	Speech recognition
English	Chinese	Machine translation
Image, Radar info	Position of other cars	Autonomous driving

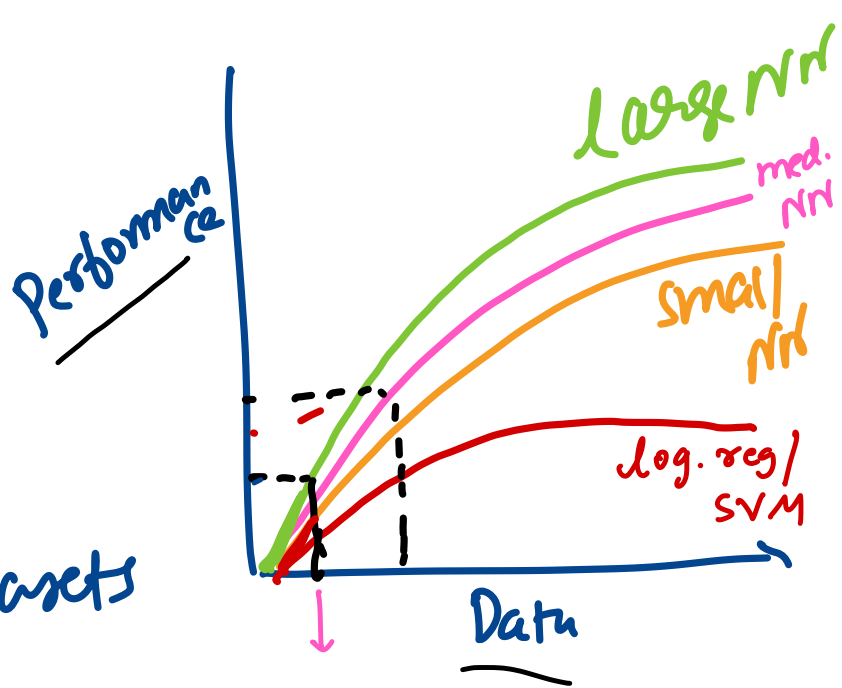
(last) NN
many inputs (features)

many outputs

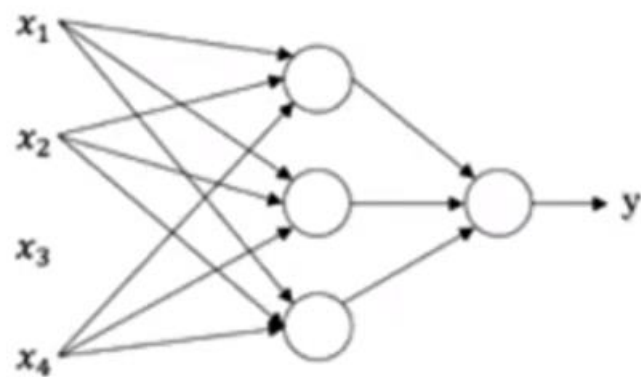
Complex RSHIP

non-linear RSHIPS

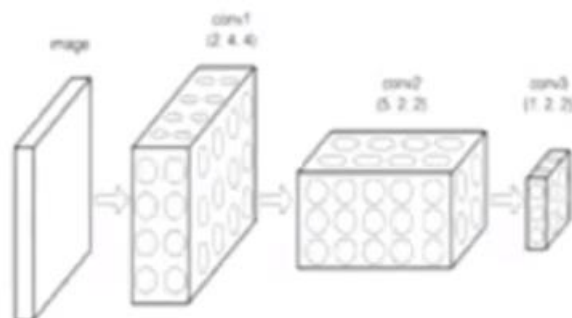
work well with large datasets



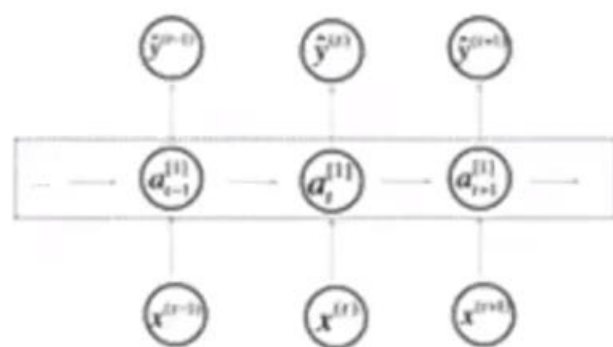
Neural Network examples



Standard NN



Convolutional NN



Recurrent NN

Supervised Learning

Structured Data

Size	#bedrooms	...	Price (1000\$s)
2104	3		400
1600	3		330
2400	3		369
⋮	⋮		⋮
3000	4		540

User Age	Ad Id	...	Click
41	93242		1
80	93287		0
18	87312		1
⋮	⋮		⋮
27	71244		1

Unstructured Data



Audio



Image

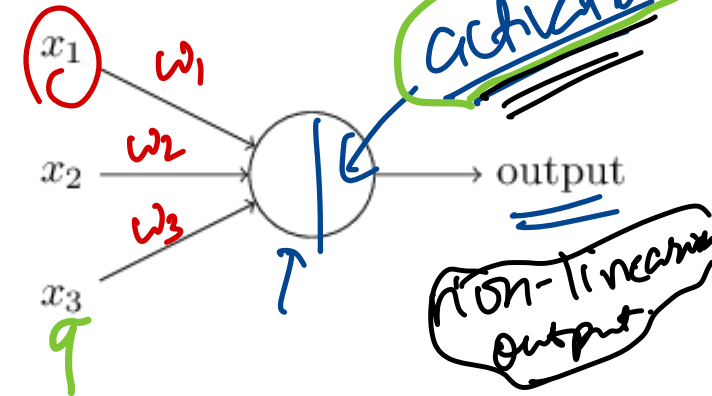
Four scores and seven
years ago...

Text

Perceptron

{ TLV Threshold Logic Unit ^{primitive perceptron}
LTV Linear Threshold Unit ^{contemporary perceptron}

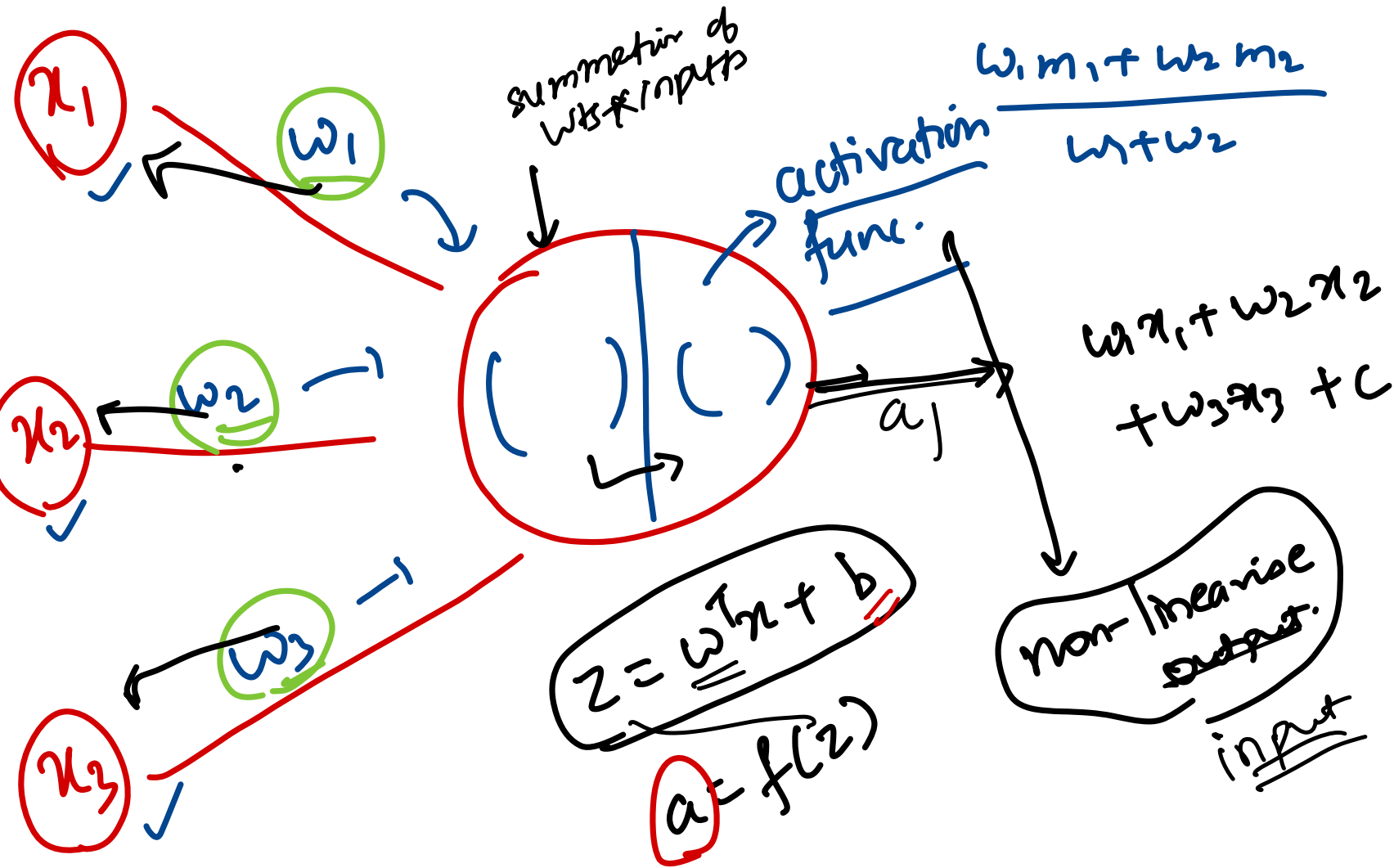
- A perceptron takes several binary inputs, $x_1, x_2, \dots$, and produces a single binary output
- Computing the output? – we introduce weights, which express the importance of an input
- By varying the weights and the threshold, we can get different models of decision-making
- Perceptron helps makes decisions by weighing up evidence!



$$\text{output} = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq \text{threshold} \\ 1 & \text{if } \sum_j w_j x_j > \text{threshold} \end{cases}$$

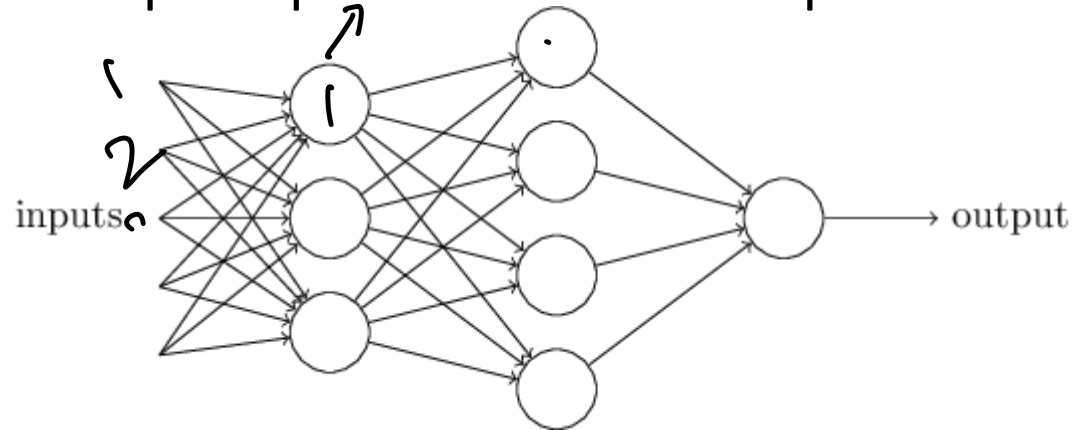
A green oval encircles the entire equation. A green '20' is written above the 'threshold' in the first case. A green double arrow points to the bottom right of the oval.

$$\sum_j w_j x_j$$



Perceptron

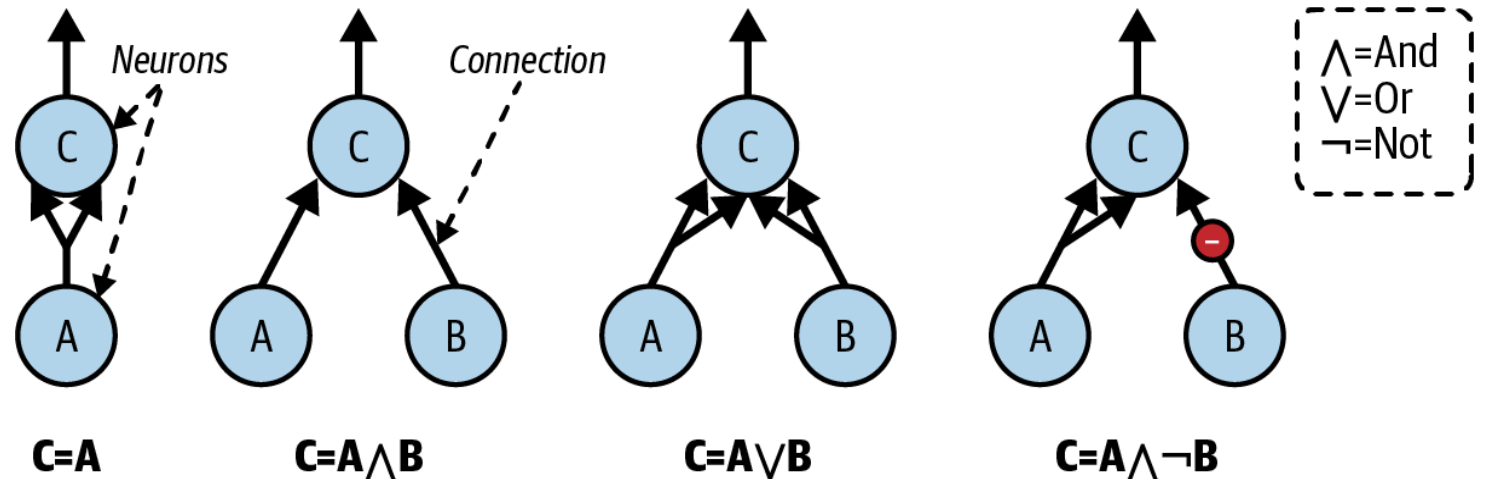
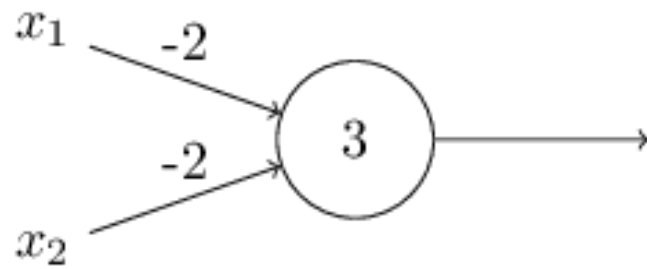
- A complex network of perceptrons could make quite subtle decisions!



- The first *layer* of perceptrons is making three very simple decisions by weighing the input evidence.
- The perceptrons in the second layer? – making decision by weighing results from the first layer, thus can make a decision at a more complex and abstract level.
- More complex decisions can be made by the perceptron in the third layer.
- In this way, a many-layer network of perceptrons can engage in sophisticated decision making.

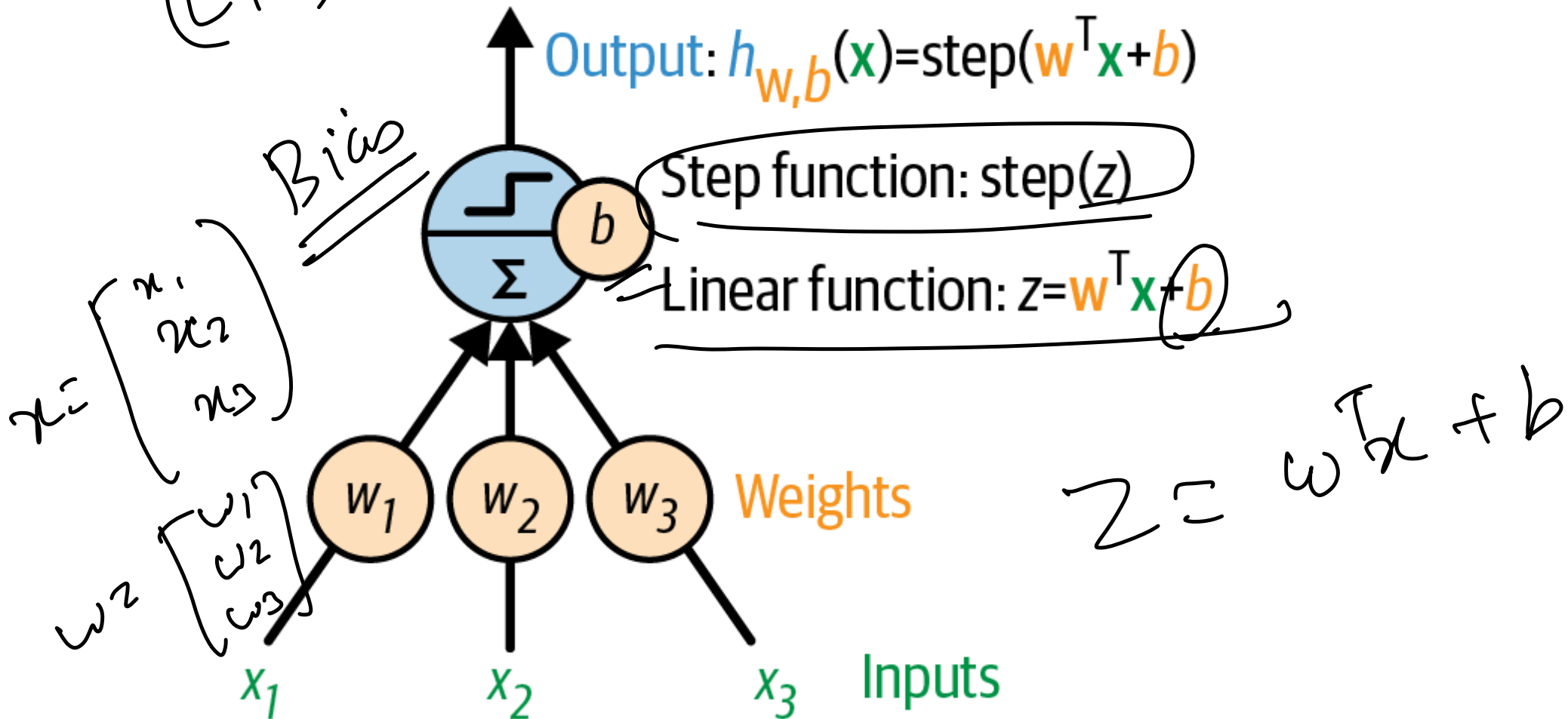
Perceptron – more uses!

- Another way perceptrons can be used is to compute the elementary logical functions, such as AND, OR, and NAND

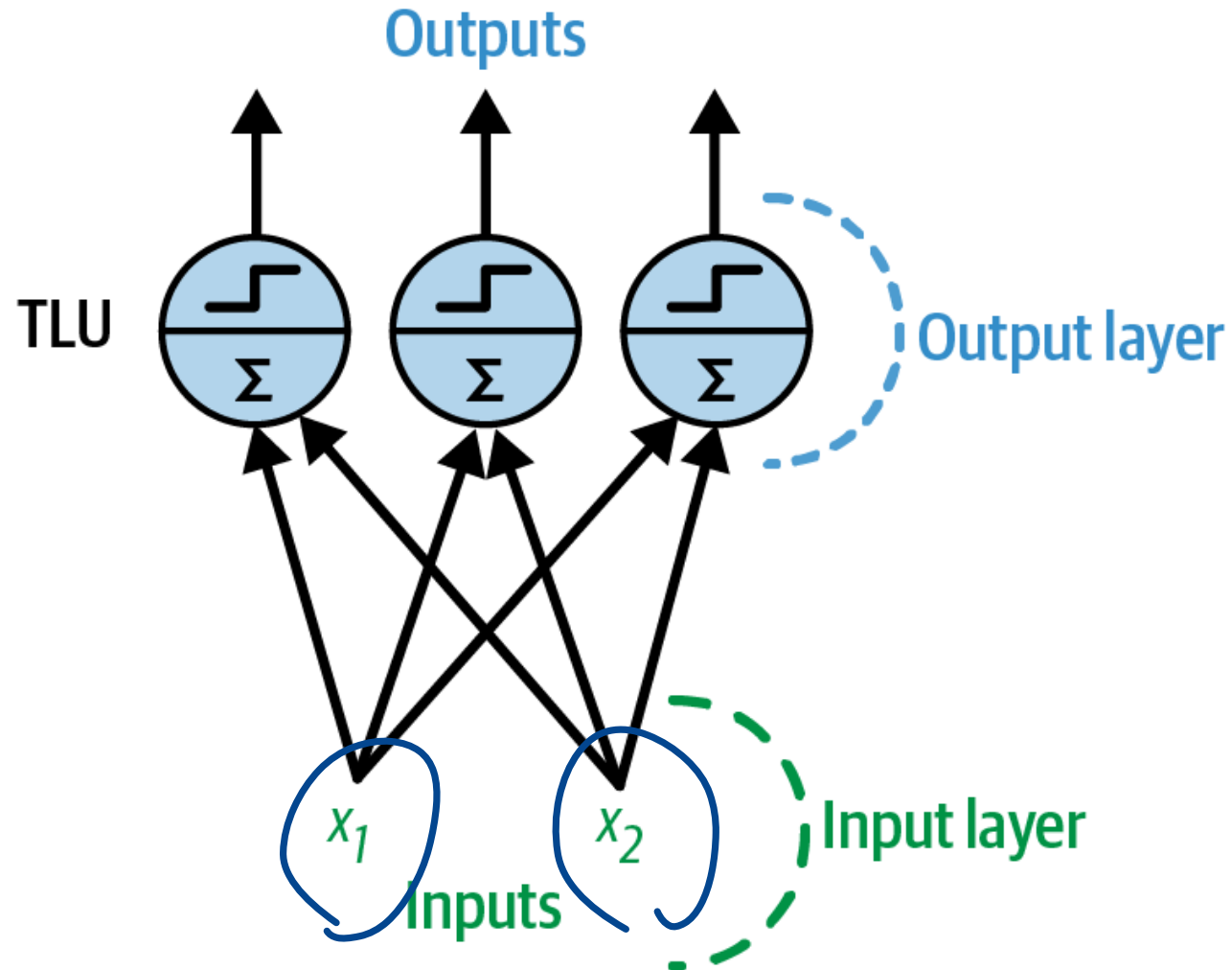


Perceptron / Threshold Logical Unit (TLU)

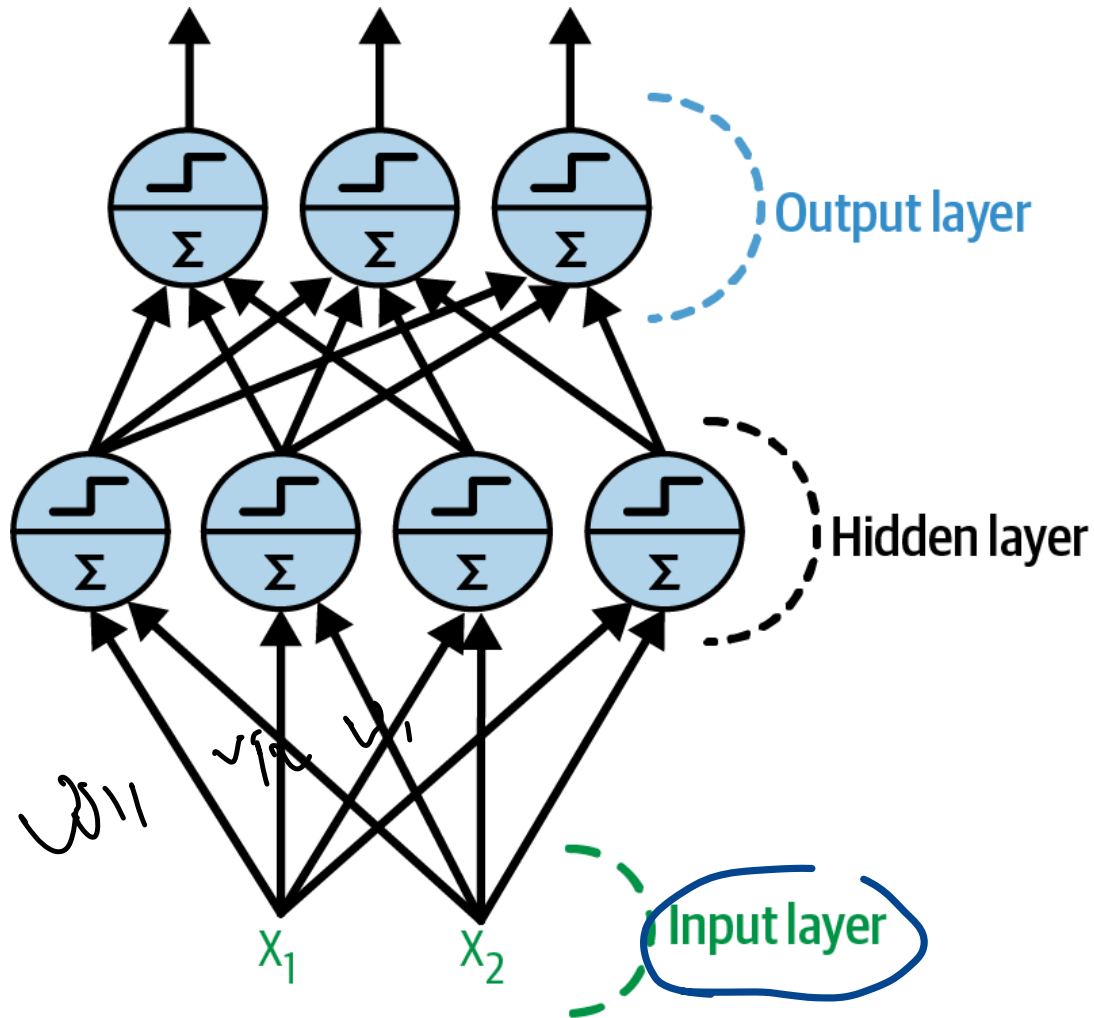
(TLU)



A perceptron with 2 inputs and 3 output neurons

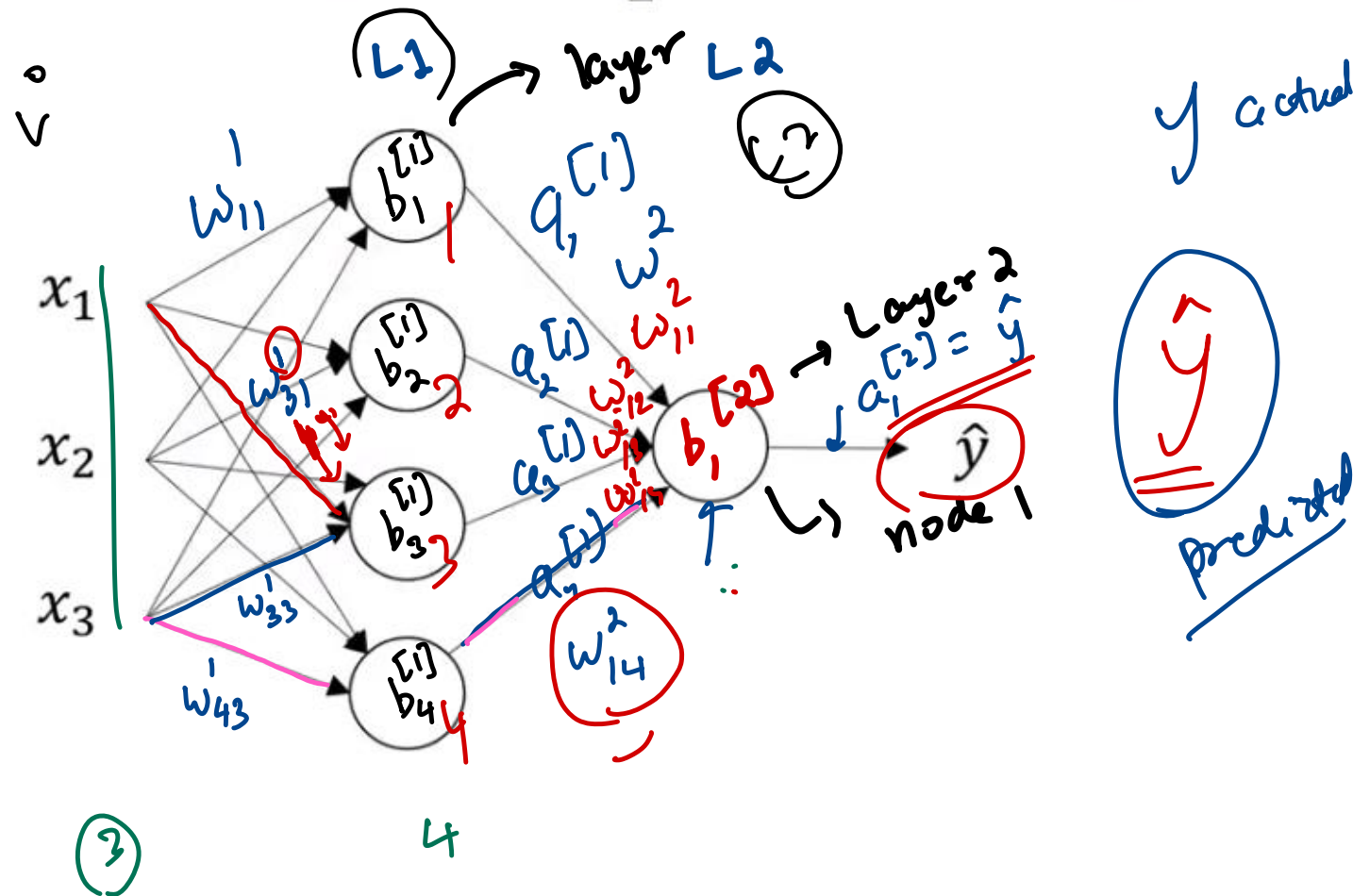


Multilayer Perceptron

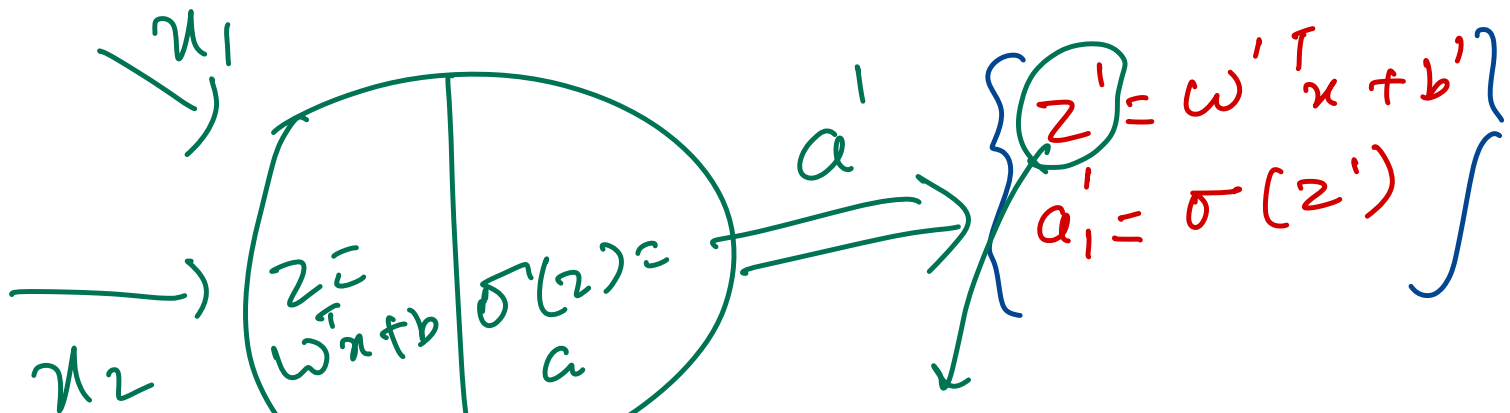


Cannot
physically
interpret
neural
networks.

Neural Network Representation



A diagram illustrating a weight W_{jk} in a neural network. The weight is represented by the symbol W with a pink circle around the subscript jk . Three arrows point from the circle to labels: a pink arrow to the left labeled "nodes", a green arrow pointing down labeled "dest. layer", and a green arrow pointing down and to the right labeled "previous layer". Above the weight, the text " $d \Rightarrow$ destination layer" is written in green.



Middle layer nodes input features

matrix sizes

$$\left\{ \begin{array}{l} W' = \begin{pmatrix} 4 & 3 \end{pmatrix} \\ b' = (4, 1) \\ z' = (4, 1) \end{array} \right. \left| \begin{array}{l} a' = (4, 1) \\ W^2 = (1, 4) \\ b^2 = (1, 1) \\ z^2 = (1, 1) \\ a^2 = (1, 1) \end{array} \right.$$

$$\begin{bmatrix} - & w_1'^T & - \\ - & w_2'^T & - \\ - & w_3'^T & - \\ - & w_4'^T & - \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1' \\ b_2' \\ b_3' \\ b_4' \end{bmatrix} =$$

w'

$$\begin{bmatrix} w_1'^T & + b_1' \\ w_2'^T & + b_2' \\ w_3'^T & + b_3' \\ w_4'^T & + b_4' \end{bmatrix} = \begin{bmatrix} z_1' \\ z_2' \\ z_3' \\ z_4' \end{bmatrix}$$

$$a' = \begin{bmatrix} a_1' \\ a_2' \\ a_3' \\ a_4' \end{bmatrix} = \sigma(z')$$

1 set of examples

Vectorize across Training Examples.

$$\underline{X^{(1)}} \Rightarrow \underline{a_1^{[2](1)}} = \underline{\hat{y}^{(1)}}$$

$$X^{(2)} \Rightarrow a_2^{2} = \hat{y}^{(2)}$$

$\vdots$

$$X^{(m)} \Rightarrow$$

$$a_{(2)}^{[2](m)} = \hat{y}^{(m)}$$

↳ node no.

layer no.
sample or example no.

ρ
 ρ
 ρ
 ρ

$$X = \begin{bmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \end{bmatrix}$$

↓ features

→ training example

$$z^{[1]} = \begin{bmatrix} z^{[1]}(1) & z^{[1]}(2) & \dots & z^{[1]}(m) \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \end{bmatrix}$$

hidden
layer
output

+ training examples

$$\underbrace{W^{[1]}}_{4 \times 3} \begin{bmatrix} x^{(1)} & \dots & x^{(m)} \end{bmatrix}_{3 \times m} = \begin{bmatrix} z^{[1]}(1) & \dots & z^{[1]}(m) \end{bmatrix}_{4 \times m} \quad (\text{w/o bias})$$

Weights + biases are same for all the
training
examples
↓
(4x1) matrix

$$z = \frac{\omega^T x + b}{4 \times m} \quad 4 \times 1$$

$4 \times (n+1)$

$$\begin{bmatrix} - & - \\ - & - \\ - & - \\ - & - \end{bmatrix} + \begin{bmatrix} - \\ - \\ - \\ - \end{bmatrix} \quad 4 \times 1$$

Weights + biases are NOT example dependent.

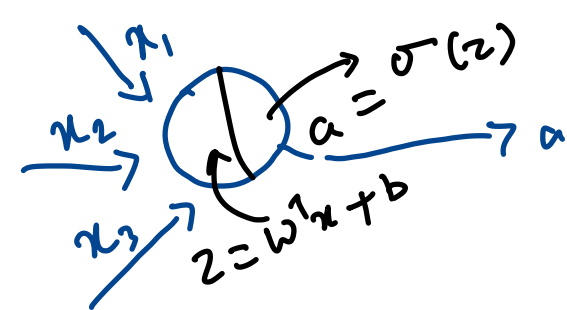
for $i = 1$ to m

$$z^{[1]}(\underline{i}) = w^{[1]} x^{(i)} + b^{[1]}$$

$$\underline{a^{[1]}(i)} = \sigma(\underline{z^{[1]}(i)})$$

$$z^{[2]}(\underline{i}) = w^{[2]} \underline{a^{[1]}(i)} + b^{[2]}$$

σ function
 σ is same for all
train example.



$$z' = w'' T x + b'$$

$$a_i^{(1)} = \sigma(z')$$

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

$$W^T \Rightarrow 4 \times 3$$

$$\begin{matrix} 3 \times 1 \\ - & - \end{matrix}$$

$$W' = (3, 4)$$

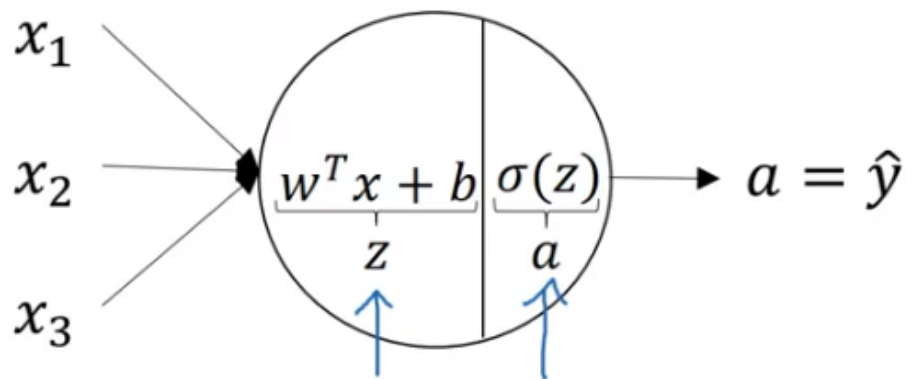
$$b' = (4, 1)$$

input

$$W^2 = (1, 4)$$

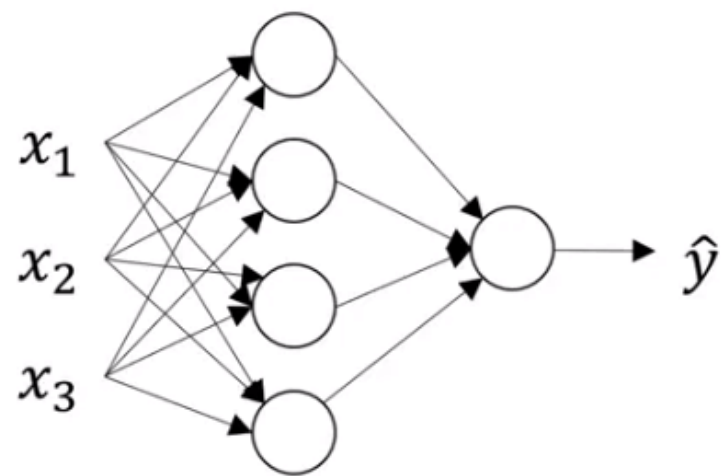
$$b^2 = (1, 1)$$

hidden nodes



$$z = w^T x + b$$

$$a = \sigma(z)$$



▶ what is the role of bias term in a neural network

ChatGPT

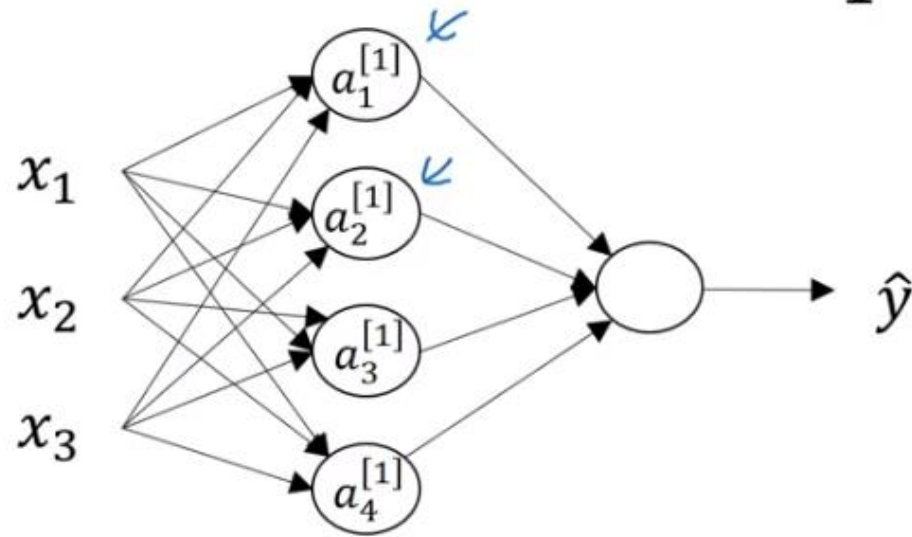
▶ The bias term in a neural network is an additional parameter added to each neuron (or node) in a layer, except for the input layer. It allows the network to capture more complex relationships between the inputs and outputs by introducing an offset or bias to the activation function.

The role of the bias term is to shift the activation function horizontally, influencing when the neuron should be activated or deactivated. Without a bias term, the activation function would only be able to pass through the origin (0,0) on the graph, which could limit the network's ability to learn certain patterns.

In practical terms, the bias term allows the neural network to better fit the training data by adjusting the decision boundary. It provides flexibility in modeling the data, enabling the network to learn patterns even when the input data is not centered around zero or when there are non-linear relationships between the features and the target.

In summary, the bias term in a neural network helps improve the network's flexibility and ability to learn complex patterns by introducing an additional parameter that shifts the activation function.

Neural Network Representation



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]}, \quad a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]}, \quad a_2^{[1]} = \sigma(z_2^{[1]})$$

$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]}, \quad a_3^{[1]} = \sigma(z_3^{[1]})$$

$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]}, \quad a_4^{[1]} = \sigma(z_4^{[1]})$$

Given input x :

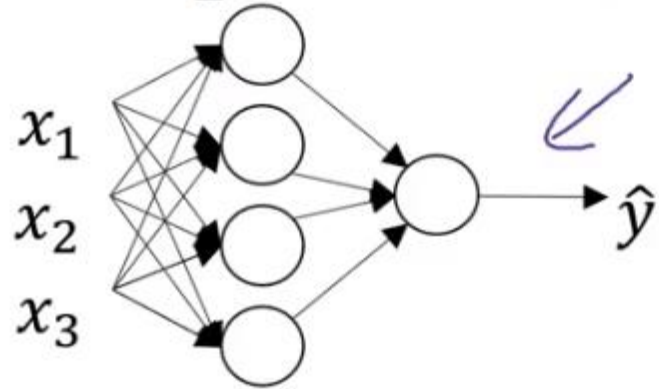
$$\rightarrow z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

Recap of vectorizing across multiple examples



$$X = \begin{bmatrix} | & | & & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & & | \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} | & | & & | \\ a^{1} & a^{[1](2)} & \dots & a^{[1](m)} \\ | & | & & | \end{bmatrix}$$

for $i = 1$ to m

$$z^{[1](i)} = W^{[1]}x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]}a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

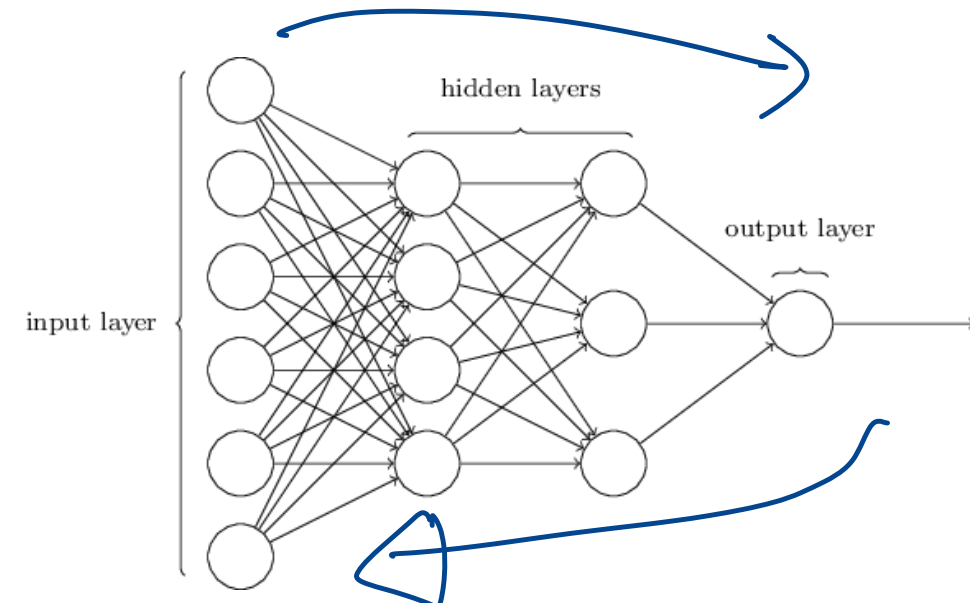
$$A^{[2]} = \sigma(Z^{[2]})$$

“Feedforward” neural networks

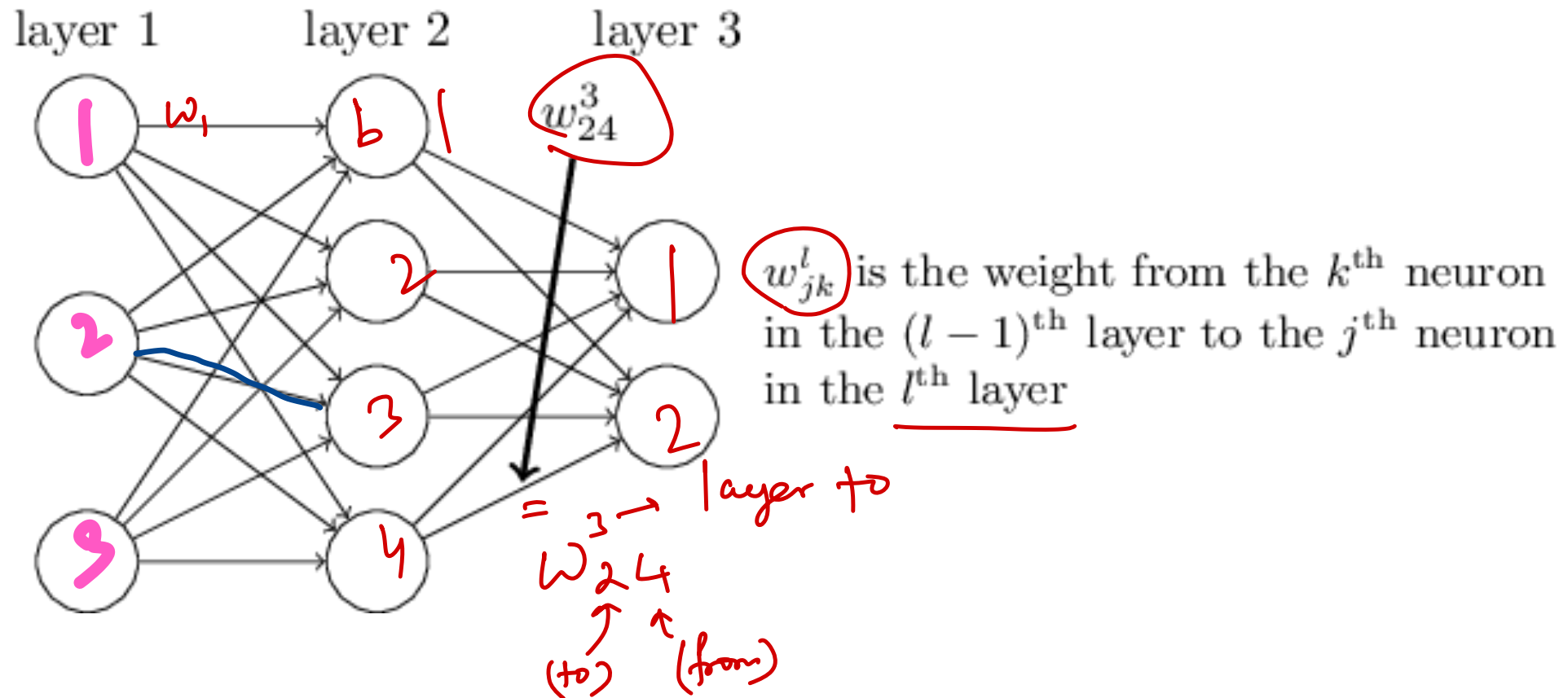
← Feedback is required!

- We studied neural networks where the output from one layer is used as input to the next layer.
- Such networks are called *feedforward* neural networks.
- This means there are no loops in the network – information is always fed forward, never fed back.

Is this network good enough? no

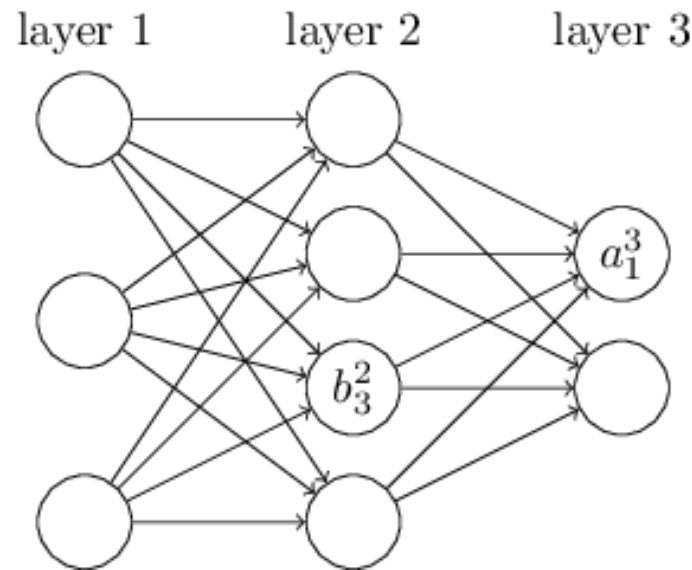


Neural Networks – notations



Neural Networks – notations

- We use a similar notation for the network's biases and activations. Explicitly, we use b_j^l for the bias of the j^{th} neuron in the l^{th} layer. And we use a_j^l for the activation of the j^{th} neuron in the l^{th} layer.



Neural Networks – notations

- So, the activation a_j^l of the j^{th} neuron in the l^{th} layer is related to the activations in the $(l-1)^{\text{th}}$ layer by the equation:

$$a_j^l = \sigma \left(\sum_k w_{jk}^l a_k^{l-1} + b_j^l \right)$$

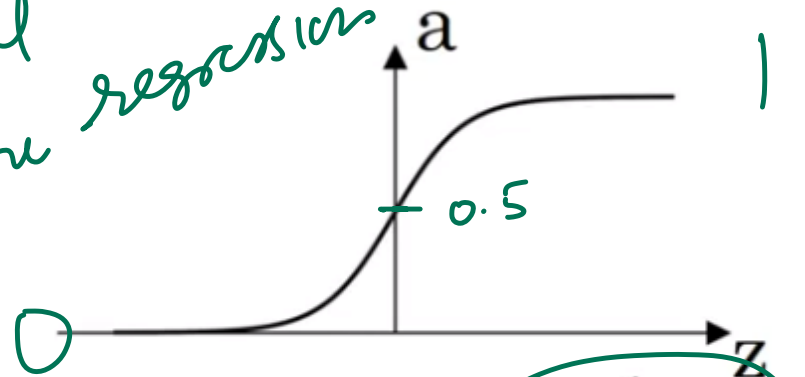
- where the sum is over all neurons k in the $(l-1)^{\text{th}}$ layer.

Neural Networks – notations

- We rewrite this expression in a matrix form.
- We define a weight matrix $\mathbf{w}^l$ for each layer, l , where the entries of the weight matrix $\mathbf{w}^l$ are the weights connecting to the l^{th} layer of neurons. That is, the entry in the j^{th} row and k^{th} column is w_{jk}^l
- Similarly, for each layer l , we define a bias vector, $\mathbf{b}^l$, where the components of the bias vector are just the values b_j^l , one component for each neuron in the l^{th} layer.
- We define an activation vector $\mathbf{a}^l$ whose components are the activations a_j^l .
- Finally: $\mathbf{a}^l = \sigma(\mathbf{w}^l \mathbf{a}^{l-1} + \mathbf{b}^l)$
- Sometimes written as: $\mathbf{a}^l = \sigma(\mathbf{z}^l)$ and $\mathbf{z}^l \equiv \mathbf{w}^l \mathbf{a}^{l-1} + \mathbf{b}^l$

Pros and cons of activation functions

Recall logistic regression

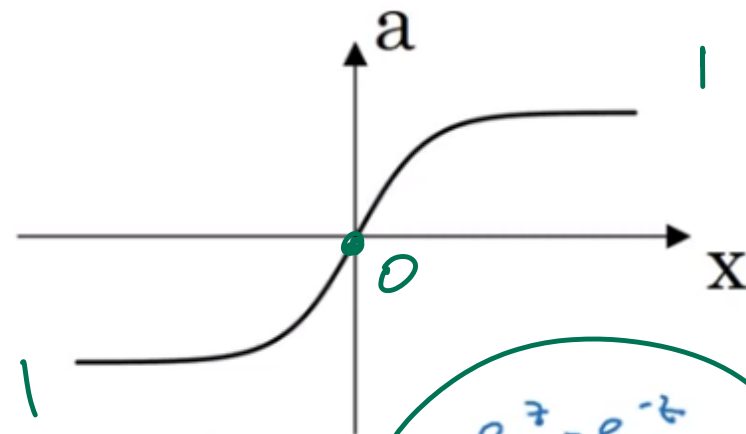


sigmoid: $a = \frac{1}{1 + e^{-z}}$

(0, 1)

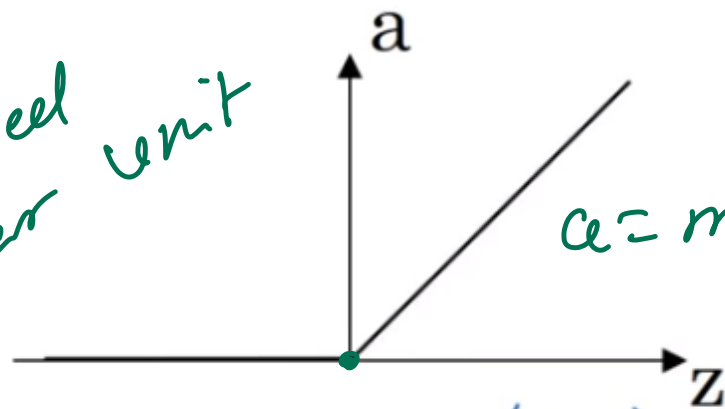
$$a = \frac{1}{1 + e^{-2}}$$

(-1, 1) - 1



tanh: $a = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

Rectified linear unit



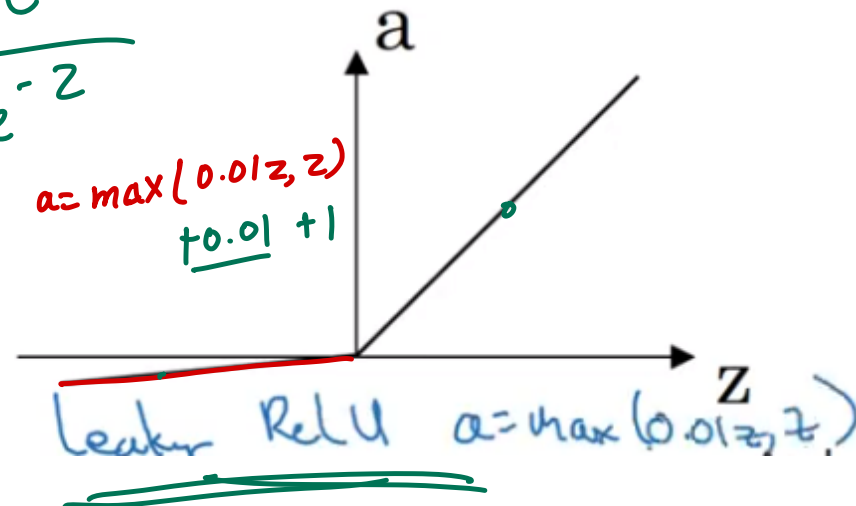
$$a = \max(0, z)$$

ReLU $a = \max(0, z)$

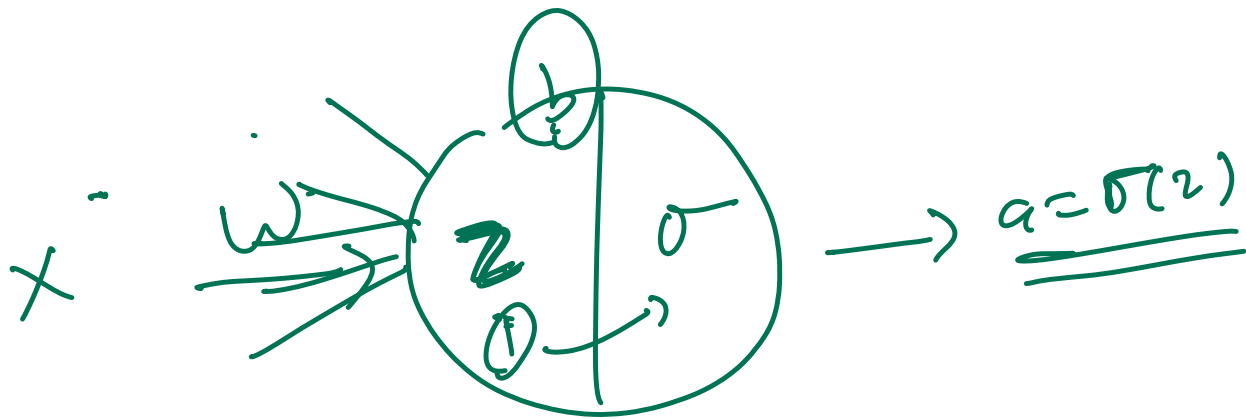
$$\frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$a = \max(0.01z, z)$$

+0.01 +1

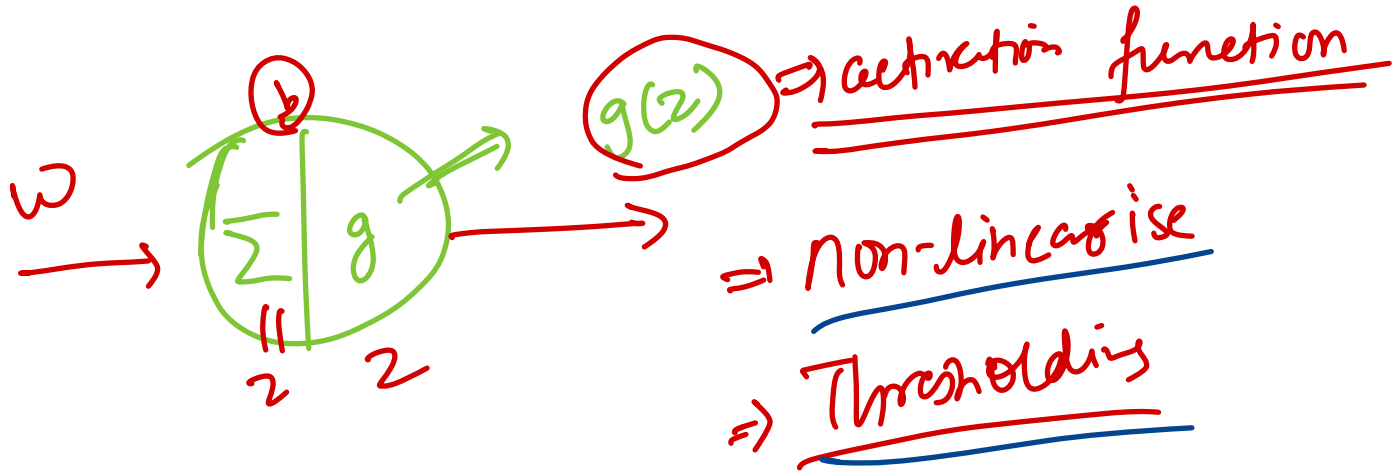


Leaky ReLU $a = \max(0.01z, z)$



non-linear

Thresholding



Why do we need non-linear activation function

$$z^{[1]} = w^{[1]}x + b^{[1]}$$

$$a^{[1]} = g(z^{[1]}) \Rightarrow a^{[1]} = z^{[1]}$$

$$z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = z^{[2]}$$

$$a^1 = z^1 = w^1x + b^1$$

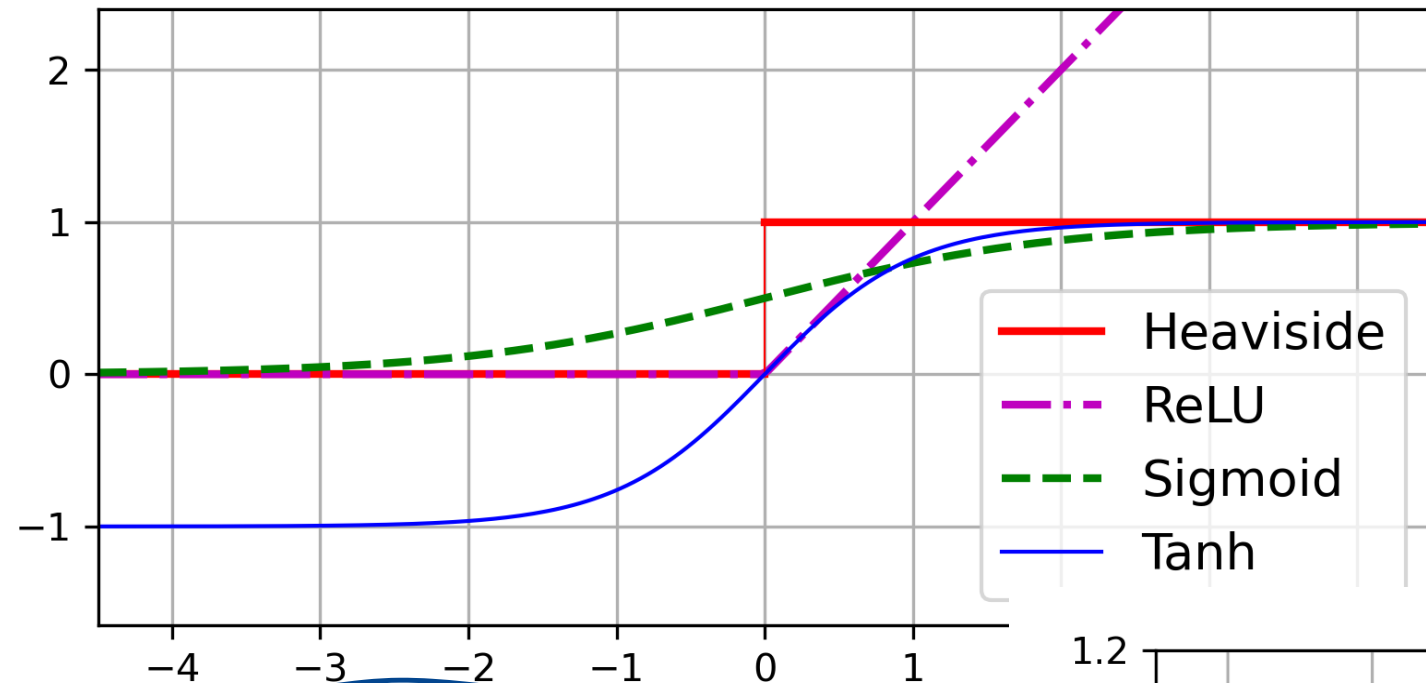
$$a^2 = w^2a^1 + b^2$$

$$a^2 = w^2(w^1x + b^1) + b^2$$

MUST be non-linear!
a linear act. func is useless.

$$a^2 = \underbrace{w^2w^1}_{b^1}x + \underbrace{w^2b^1 + b^2}_{b^1}$$

Activation functions

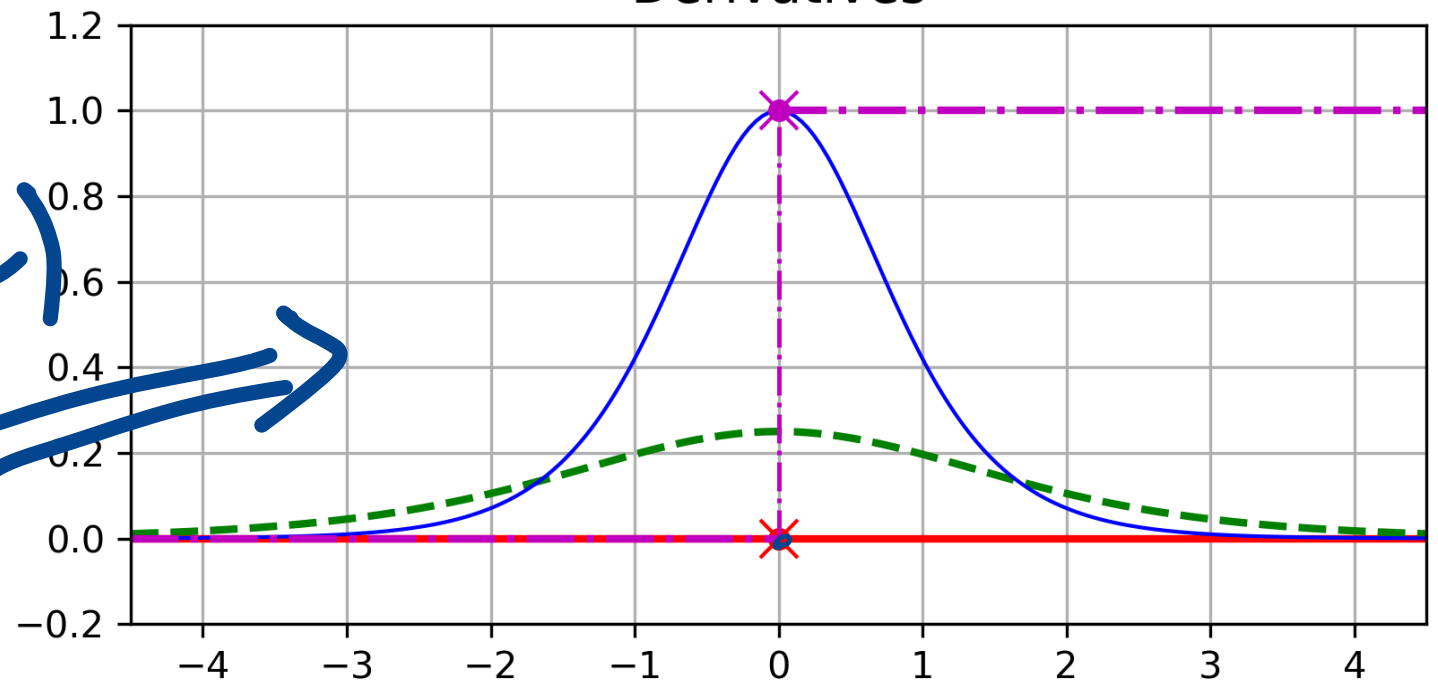


Heaviside Act Func.

$$\sigma(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Useful in Backpropagation

Derivatives

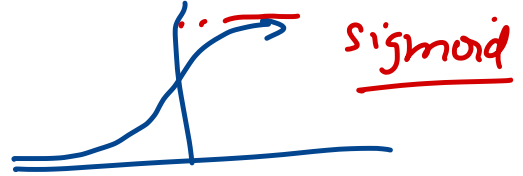


$$\frac{d g(z)}{dz} = g'(z)$$

$$g'(z) = \frac{d}{dz} g(z) \Rightarrow \text{slope @ } z$$

$$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right)$$

$$g'(z) = g(z)(1 - g(z))$$



Sanity Check:

$$z \simeq 10 \Rightarrow g(z) \simeq 1$$

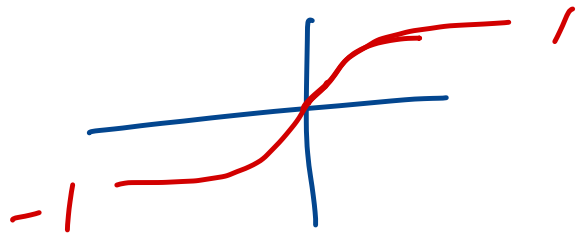
$$g'(z) \simeq 0$$

$$z=0 \quad g(z) = 1/2$$

$$g'(z) = 1/4$$

Tanh

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh(z)$$



$$g'(z) = 1 - (\tanh(z))^2$$

Sanity check:

$$z = 10$$

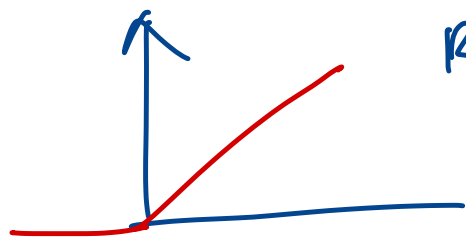
$$\tanh(z) \approx 1 \Rightarrow g'(z) \approx 0$$

$$z = -10$$

$$\tanh(z) \approx -1 \Rightarrow g'(z) \approx 0$$

$$z = 0$$

$$\tanh(z) = 0 \Rightarrow g'(z) = 1$$



ReLU $g(z) = \max(0, z)$

$$g'(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

$z < 0$
 $g(z) \rightarrow \infty$

ReLU

Leaky ReLU

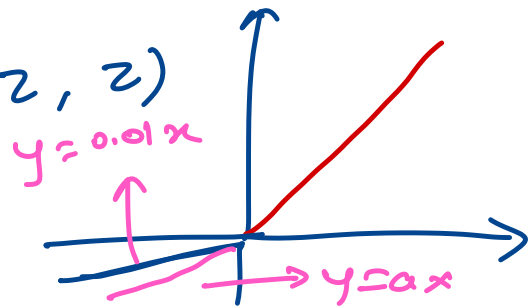
$$g(z) = \max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

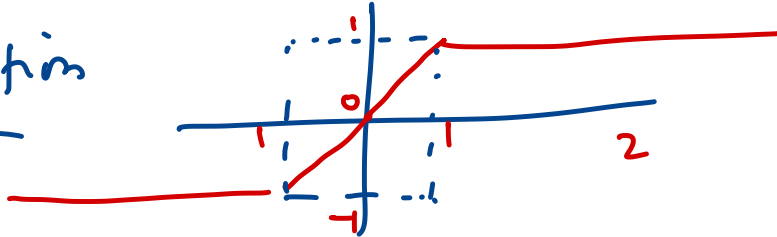
Parametric ReLU

$$g(z) = \max(az, z)$$

$z < 0 \quad g(z) \rightarrow \infty$



Satlin Function



$$g(z) = z, -1 \leq z \leq 1$$

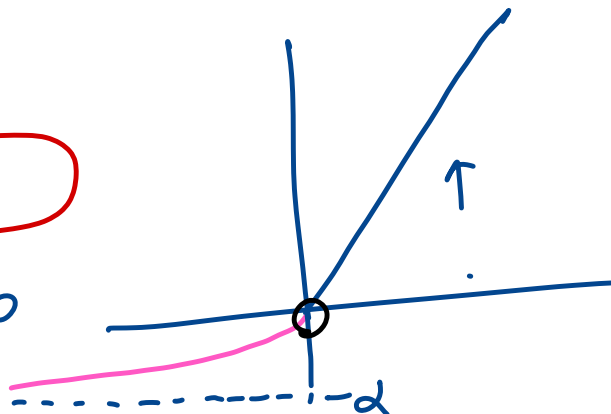
$$-1, z \leq -1$$

$$1, z \geq 1$$

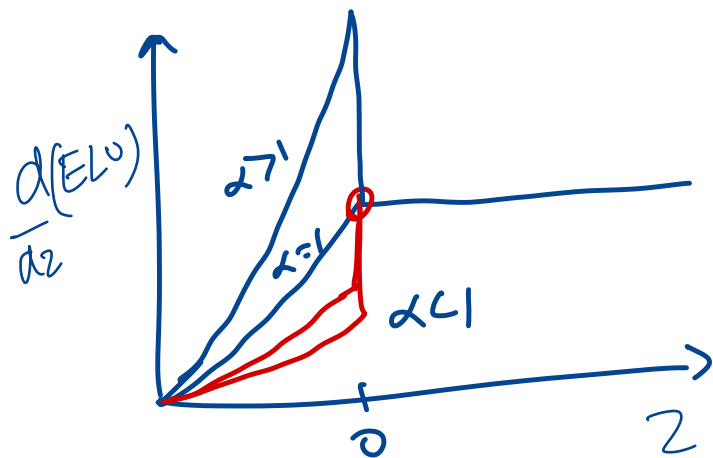
Exponential Linear Unit

$$g(z) = d(e^z - 1) \quad z < 0$$

$$z \geq 0$$



Converges to $-d$ as z becomes -vely large!



$$\frac{d}{dz} ELU(\alpha z)$$

Derivative not continuous
except @ $\alpha = 1$

Continuous

Continuously Differentiable ELU

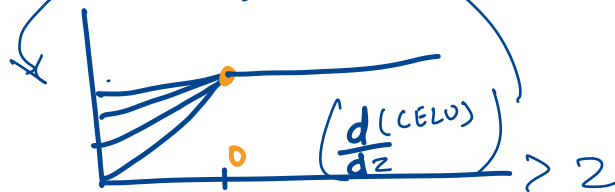
$$ELU(z, \alpha) =$$

$$z \quad \text{if } z \geq 0$$

$$d(e^{(z/\alpha)} - 1) \quad \text{if } z < 0$$

$$\frac{d}{dz} (ELU(z, \alpha)) =$$

$$e^{(z/\alpha)}$$



Sigmoid: Classification problems

- ↳ slow convergence
- ↳ not preferred for deep NN
- ↳ killing gradients. (Vanishing gradients)

input $\Rightarrow 2$ $w = 2.3$ bias $= -3$

output = ?

$$2 \times 2.3 - 3 = 1.6$$

$$\frac{1}{1 + e^{-1.6}} = 0.832$$

$$\frac{1}{1 + e^{-2}}$$

Multi-class Classifier

$$e^{z_j}$$

$$\frac{\sum_{k=1}^K e^{z(k)}}{K}$$

softmax
act. func.

$$\begin{bmatrix} 1.2 \\ 0.9 \\ 0.4 \end{bmatrix} \Rightarrow \text{softmax}$$

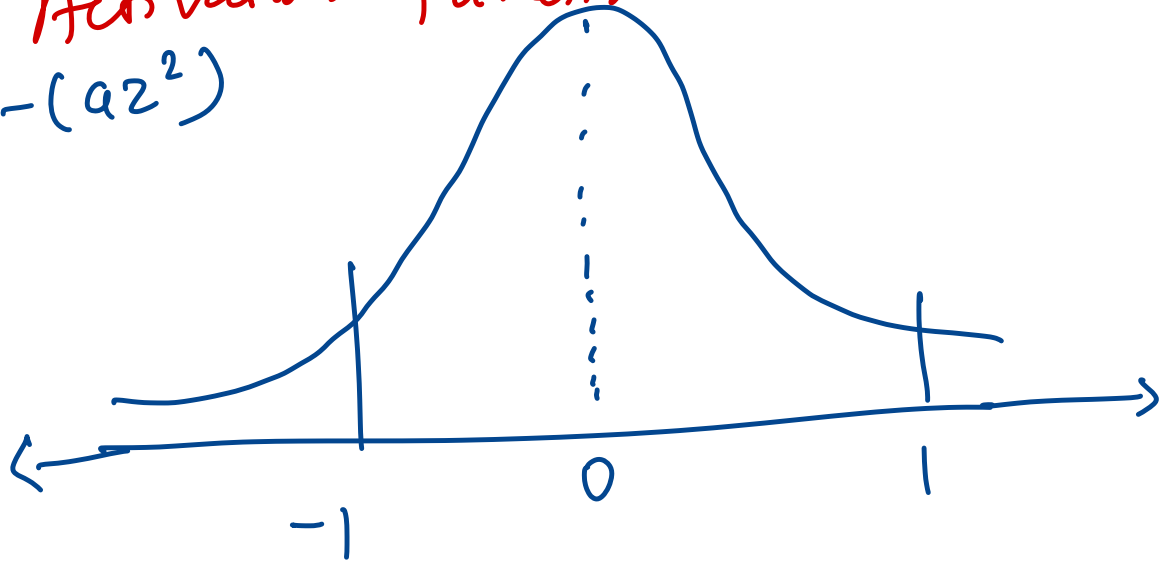
$$\begin{pmatrix} 0.46 \\ 0.34 \\ 0.2 \end{pmatrix}$$

$$\exp(1.2)$$

$$\frac{\exp(1.2) + \exp(0.9) + \exp(0.4)}{3}$$

Gaussian Activation Function

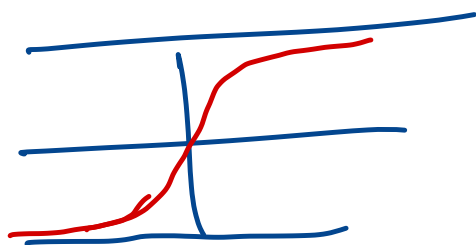
$$f(z) = e^{-(az^2)}$$



$$g'(z) = \frac{d}{dz} g(z) \Rightarrow \text{slope @ } z$$

$$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right)$$

$$g'(z) = g(z)(1-g(z))$$



Sigmoid
func.

Sanity check

$$z \approx 10 \quad g(z) \approx 1$$

$$g'(z) = 1(1-1) \approx 0$$

$$z = 0 \Rightarrow g(z) = 1/2$$

$$g'(z) = 1/4$$

Sigmoid Func.

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh.$$

$$g'(z) = 1 - (\tanh(z))^2$$

Sanity check

$$z = 10$$

$$\tanh(z) \approx 1$$

$$z = -10$$

$$\tanh(z) \approx -1$$

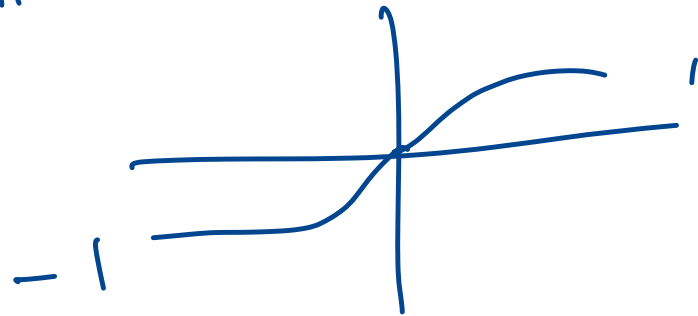
$$z = 0$$

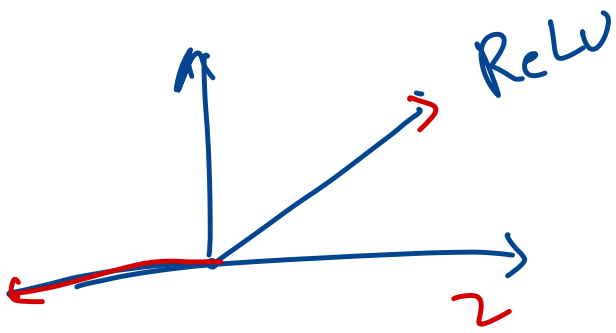
$$\tanh(z) = 0$$

$$g(z) \approx 0$$

$$g'(z) \approx 0$$

$$g'(z) = 1$$

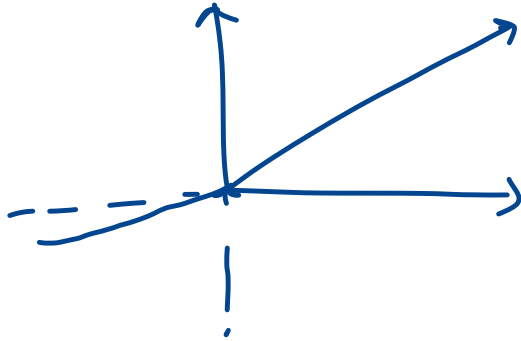




$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

Leaky ReLU



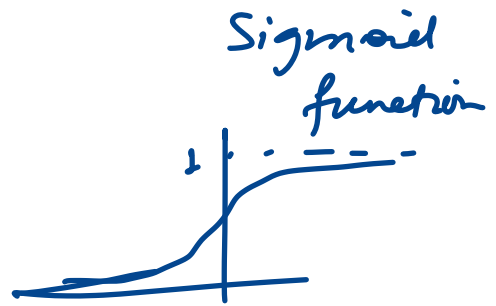
$$g(z) = \max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

$$g'(z) = \frac{d}{dz} g(z) \Rightarrow \text{slope at } z$$

$$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}} \right)$$

$$g'(z) = g(z) (1 - g(z))$$



Sanity check.



$$z \simeq 10 \quad g(z) \simeq 1$$

$$g'(z) \simeq 1(1-1) \simeq 0$$

$$z=0 \quad g(z) = 1/2$$

$$g'(z) \simeq 1/4$$

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \tanh$$

$$g'(z) = 1 - (\tanh(z))^2$$

Sanity Check:

$$z = 10 \quad \tanh(z) \simeq 1$$

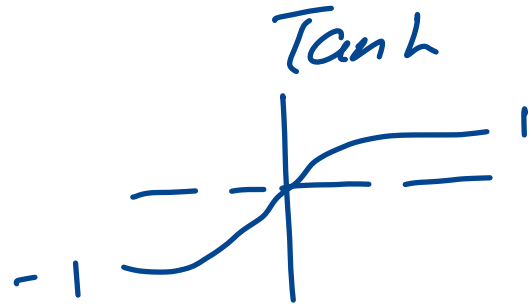
$$\Rightarrow g'(z) \simeq 0$$

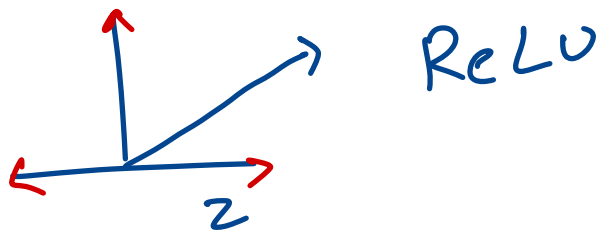
$$z = -10 \quad \tanh(z) \simeq -1$$

$$\Rightarrow g'(z) \simeq 0$$

$$z = 0 \quad \tanh(z) \simeq 0$$

$$g'(z) = 1$$



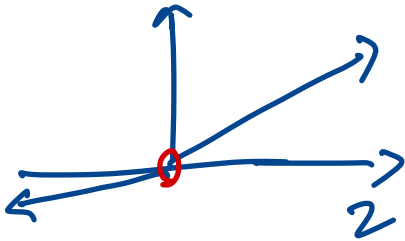


ReLU

$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}$$

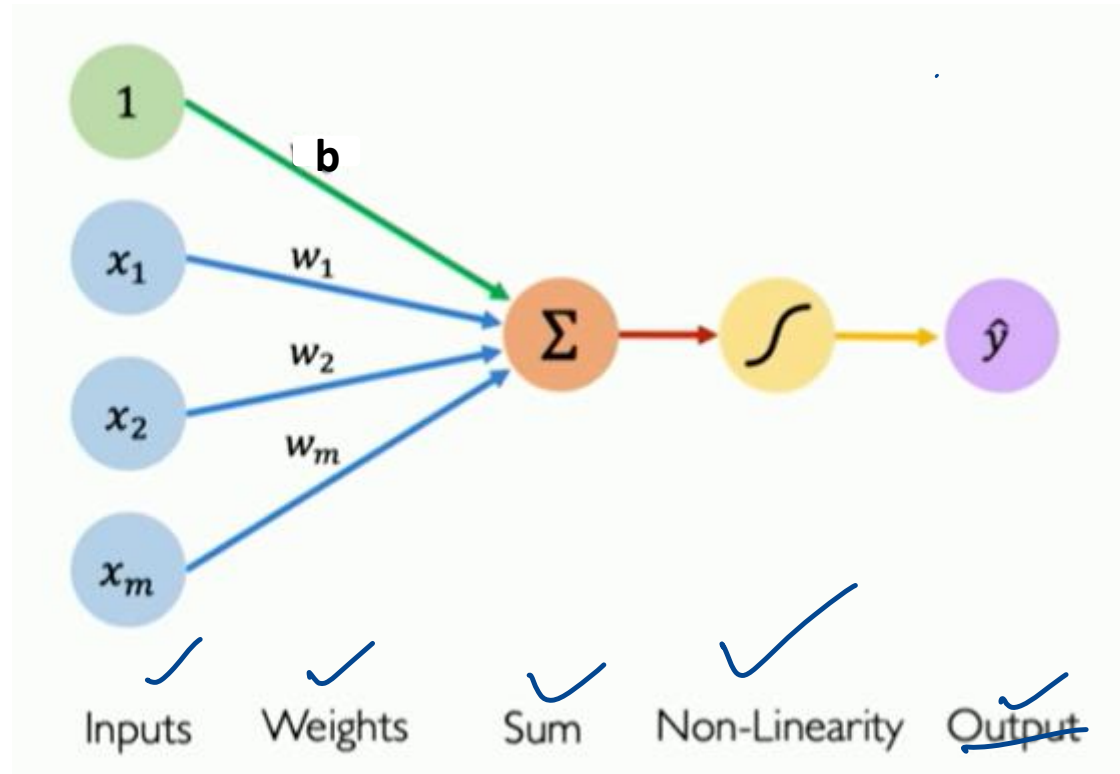
Leaky ReLU



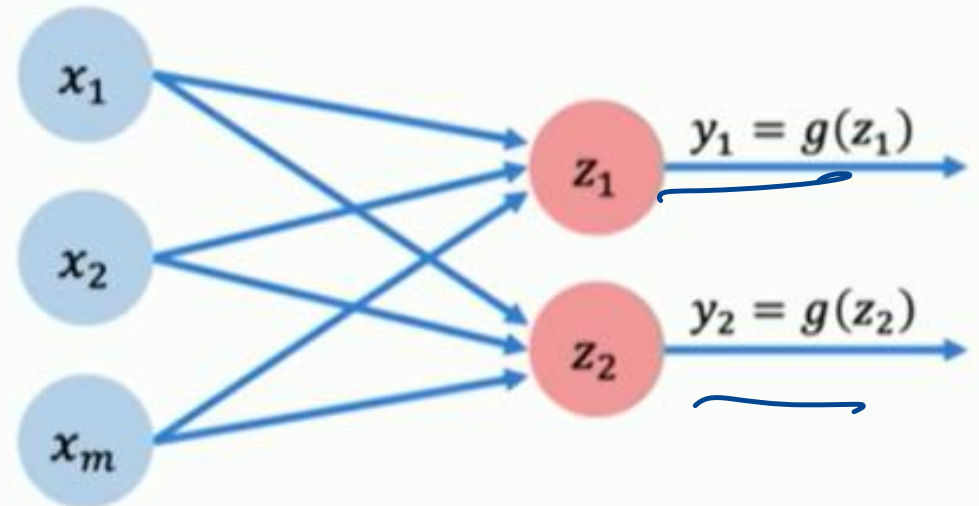
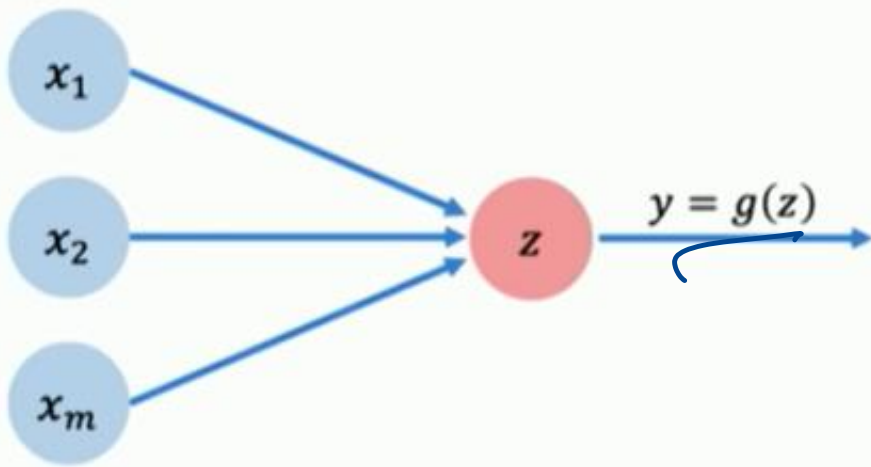
$$g(z) = \max(0.01z, z)$$

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

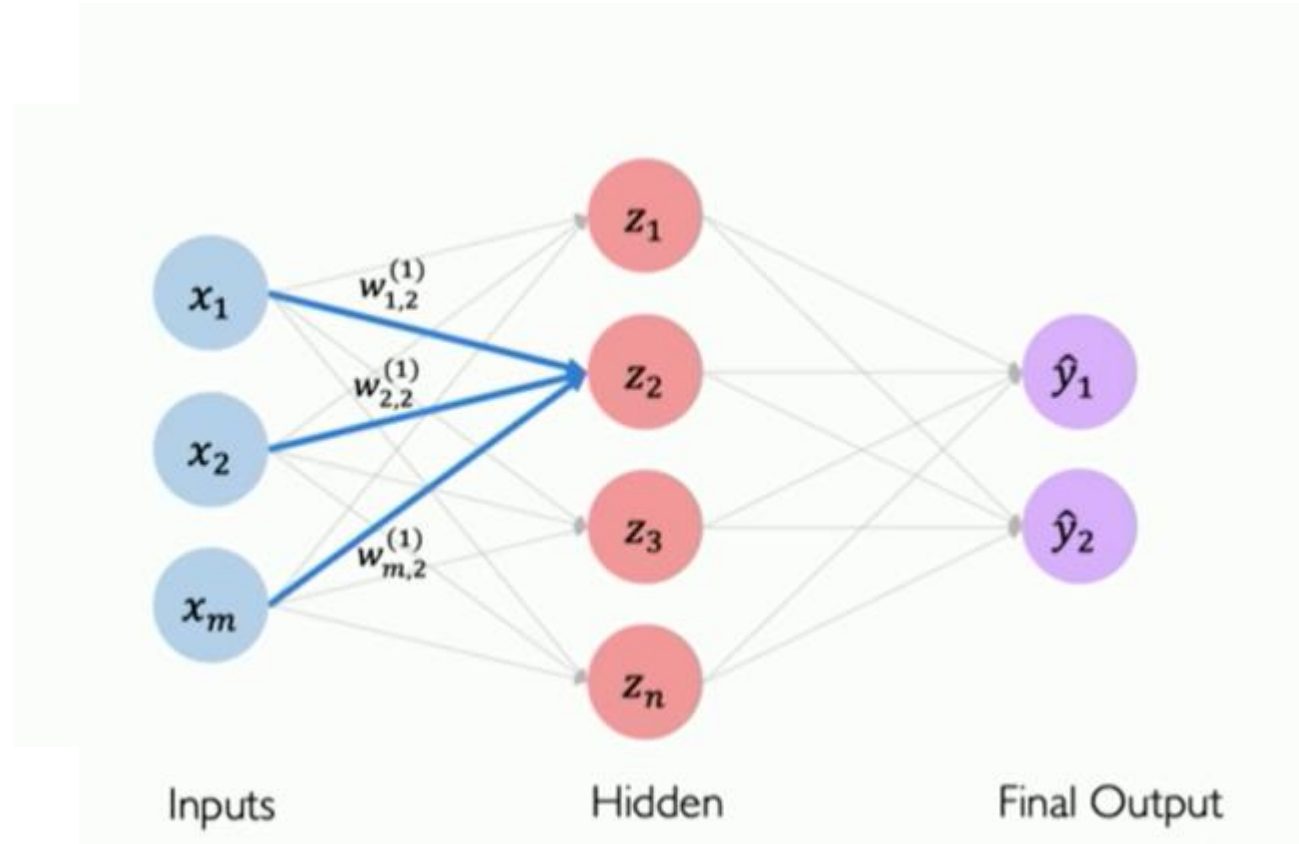
Putting the pieces together...



Putting the pieces together...

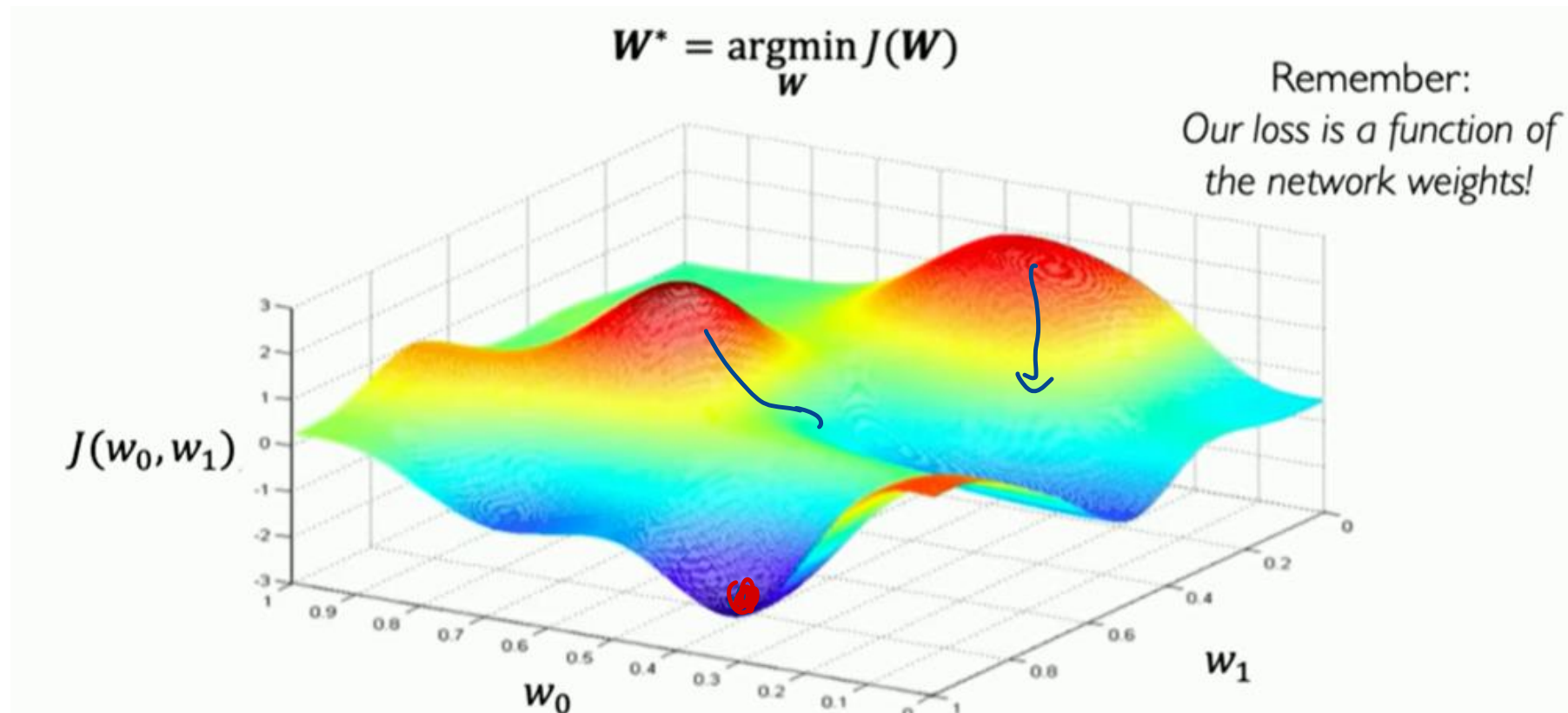


↓



Loss optimization

- We want to reach the lowest point on the loss landscape!

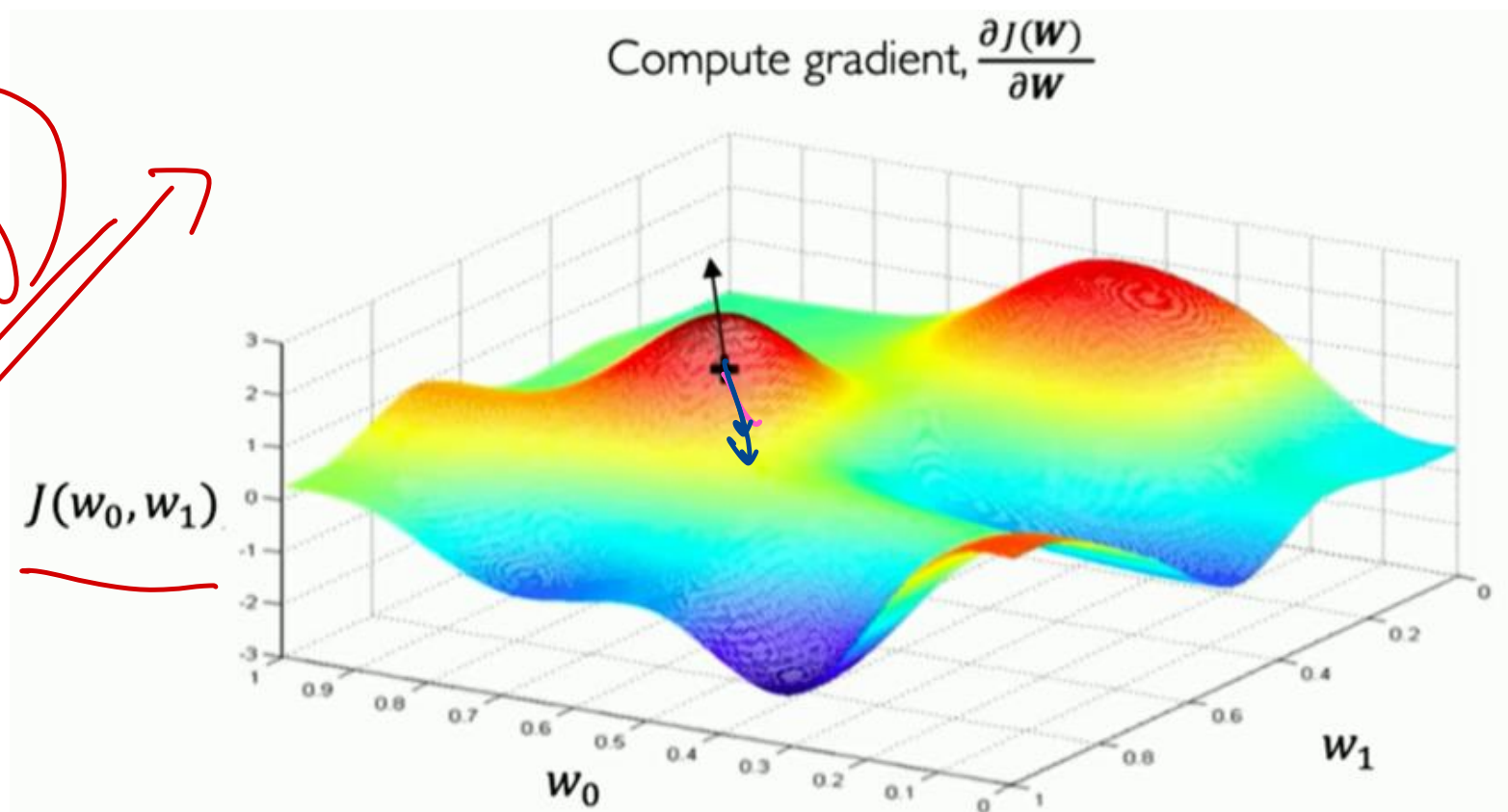


Loss optimization

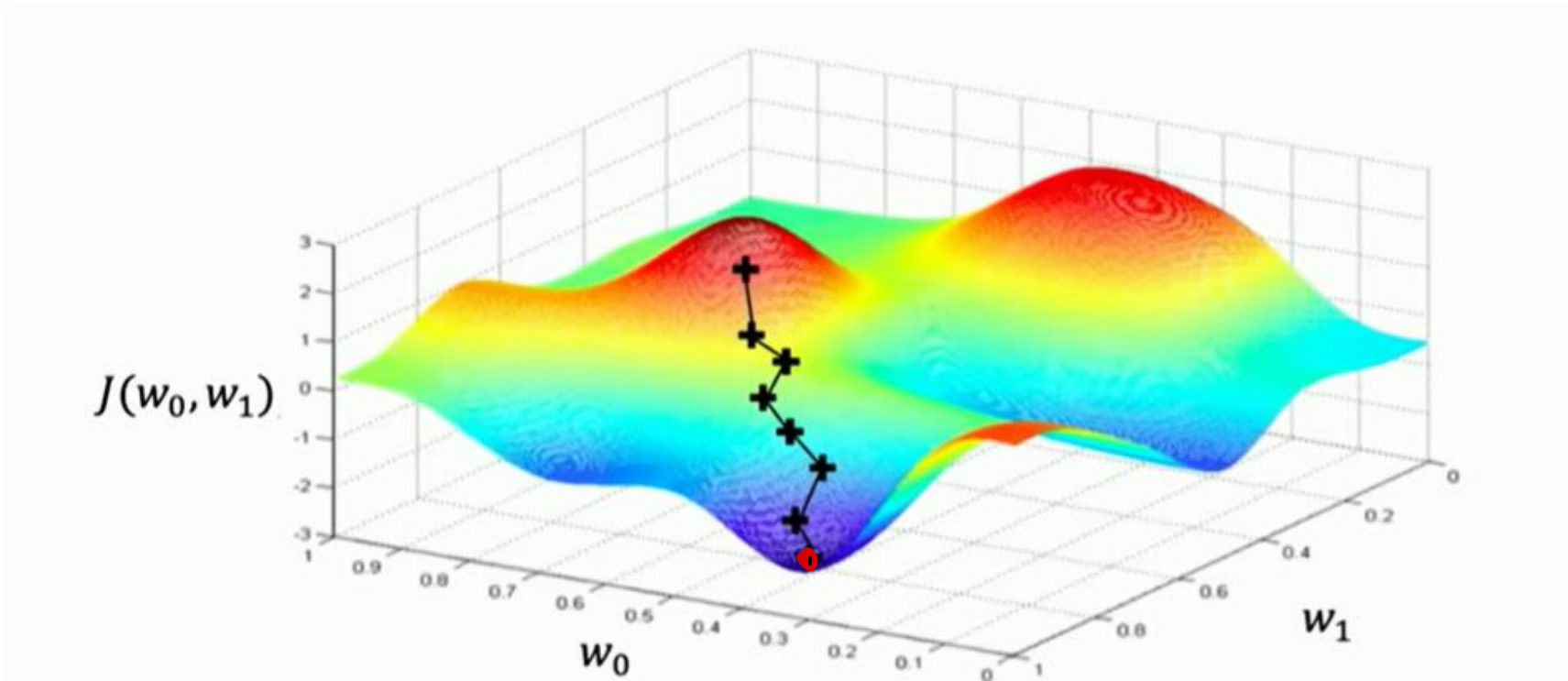
$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Handwritten notes in blue and red ink show the chain rule decomposition of the gradient, with additional scribbles and arrows indicating the flow of information.

- Take a step in the opposite direction of the gradient
- Repeat until convergence



- Repeat until convergence

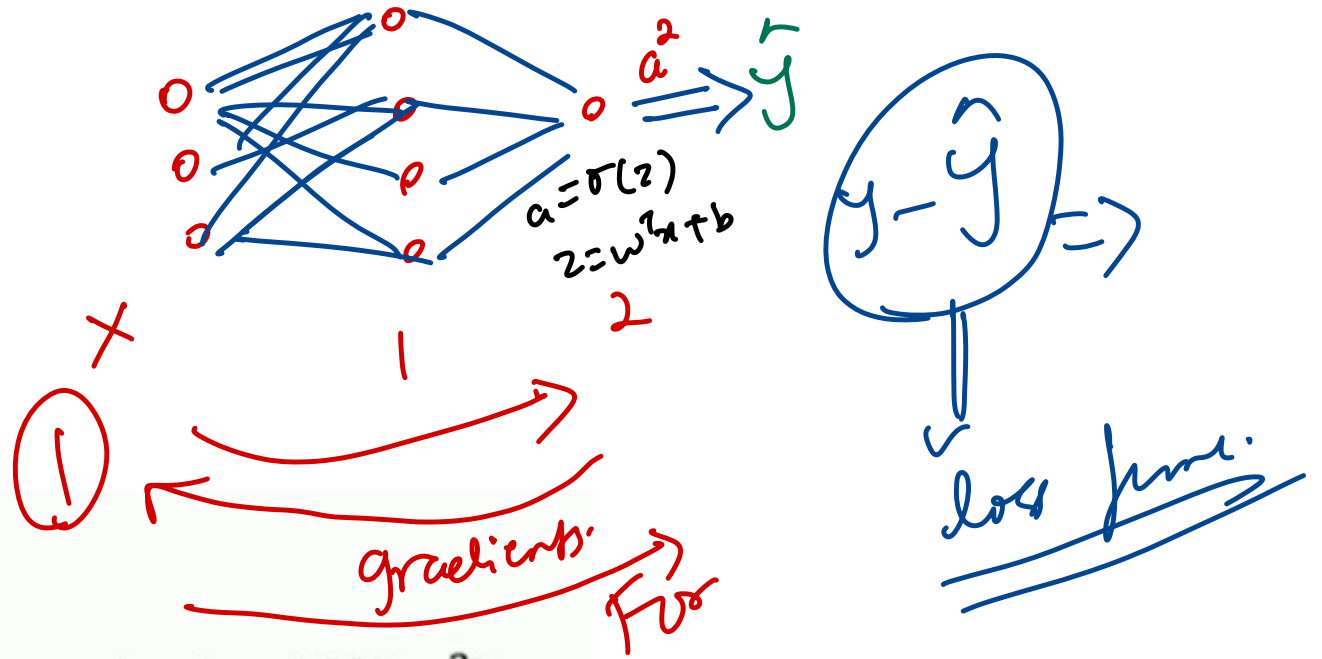


Gradient descent

$$w^{\phi} = w^0 - \eta \frac{\partial J}{\partial w}$$

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(w)}{\partial w}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(w)}{\partial w}$
5. Return weights



$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Gradient descent

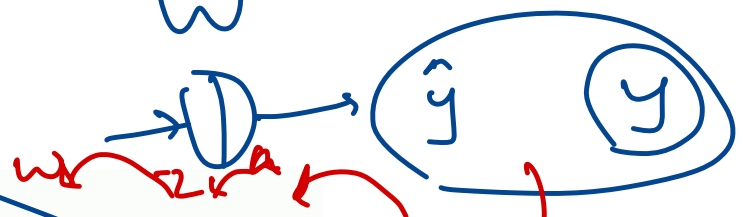
$J \Rightarrow \text{cost func.}$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$a = g(z)$
 $z = w^T x + b$

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

$$W^{\text{new}} = W^{\text{old}} - \eta \frac{\partial J(W)}{\partial W}$$


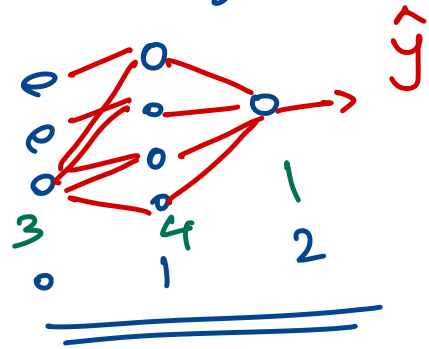
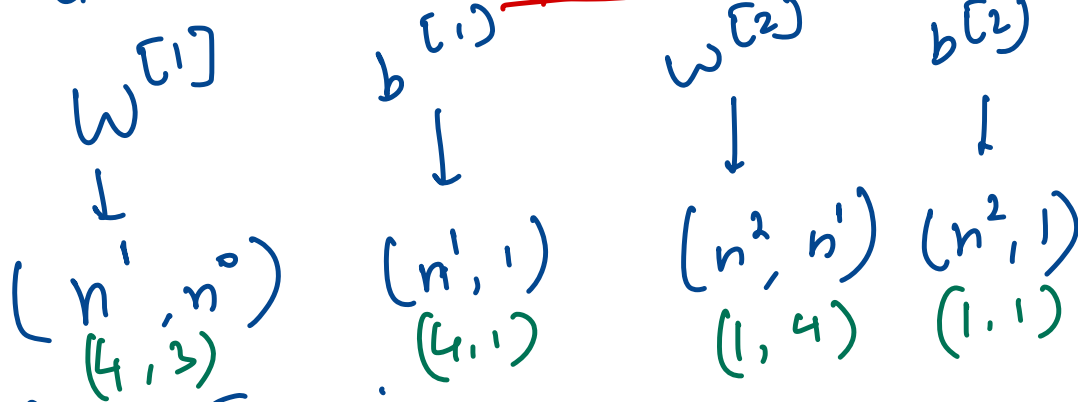
$\hat{y} \Rightarrow y$

$f(J)$

$\hat{y} - y$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Grad Descent BPA (Back propagation Algo)



Cost Function

$$J(w^1, b^1, w^2, b^2) = \frac{1}{m} \sum L(\hat{y}, y)$$

Predicted output $\hat{y}$
 labelled output y
 loss function

$$\frac{\partial J}{\partial w}$$

Repeat until convergence:

$d = l^{\text{th}}$ layer

$$dW^{[d]} = \frac{\partial J}{\partial w^{[d]}}$$

$$; db^{[d]} = \frac{\partial J}{\partial b^{[d]}}$$

$$dW^{[1]} = \frac{\partial J}{\partial w^{[1]}}$$

$$db^{[1]} = \frac{\partial J}{\partial b^{[1]}}$$

... and for other layers too.

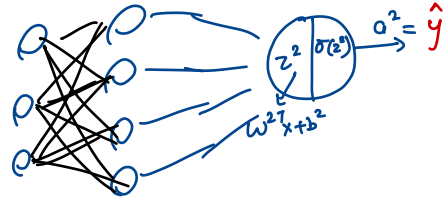
$$\text{new } W^{[1]} = \text{old } W^{[1]} - \eta \underline{dW^{[1]}} \Rightarrow \text{derivative w.r.t cost func}$$

$$\text{new } b^{[1]} = \text{old } b^{[1]} - \eta db^{[1]} \Rightarrow \text{derivative w.r.t cost func.}$$

$$w^{[2]} = w^{[2]} - \eta dw^{[2]}$$

$$b^{[2]} = b^{[2]} - \eta db^{[2]}$$

$$J(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m L^{(i)}(\hat{y}, y) \quad \text{discretely}$$



$$J(w^1, b^1, w^2, b^2) \quad \text{indirect dependence}$$

Assume Binary Classification

$$L^{(i)} = \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

$$\hat{y} = a^2$$

$$w^l = w^l - \eta \frac{\partial J}{\partial w^l}$$

$$b^l = b^l - \eta \frac{\partial J}{\partial b^l}$$

$$\frac{\partial J}{\partial w^{[2]}} = \frac{\partial J}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial w^{[2]}}$$

Chain rule

$$\frac{\partial J}{\partial w^1} = \frac{\partial J}{\partial z^{[2]}} \cdot \frac{\partial z^{[2]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial z^{[1]}} \cdot \frac{\partial z^{[1]}}{\partial w^{[1]}}$$

Forward

$$z^{[1]} = W^{[1]T} x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]T} a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

↓
sigmoid function if
binary classification

Back Propagation

$$\textcircled{1} \quad \underline{dz^{(2)}} = \frac{\partial J}{\partial z^{(2)}} = a^{(2)} - \textcircled{y} \text{ ground truth.}$$

$$\textcircled{2} \quad dw^{(2)} = \frac{1}{m} dz^{(2)} a^{(1)T}$$

$$\textcircled{3} \quad db^{(2)} = \frac{1}{m} \text{np.sum}(dz^{(2)}, \text{axis}=1, \text{keepdims}=\text{True})$$

horizontal summation
keeps output as a vector

$$\textcircled{4} \quad dz^{[1]} = W^{[2]T} dz^{[2]} \quad \textcircled{4} \quad g^{[1]'}(z^{[1]})$$


derivative

$$\textcircled{5} \quad dW^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

element wise operator

$$\textcircled{6} \quad db^{[1]} = \frac{1}{m} \text{np.sum}(dz^{[1]}, \text{axis}=1, \text{Keepdims}=\text{True})$$

Grad Descent

$w^{[1]}$



$(\underline{n'}, \underline{n^0})$

$b^{[1]}$



$(n', 1)$

BPA

$w^{[2]}$

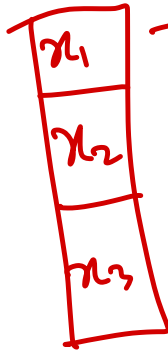


(n^2, n^1)

$b^{[2]}$



$(n^2, 1)$



b'

ϕ

ϕ

ϕ

ϕ

ϕ

ϕ

ϕ

x_2



Cost func.

$J(w', b', w^2, b^2)$

bias

loss func.

weights

$= \frac{1}{m} \sum$

$L(\hat{y}, y)$

predicted out

target



$$dw^{[1]} = \frac{\partial J}{\partial w^{[1]}}$$

$$db^{[1]} = \frac{\partial J}{\partial b^{[1]}}$$

$$dw^{[2]} = \frac{\partial J}{\partial w^{[2]}}$$

$$db^{[2]} = \frac{\partial J}{\partial b^{[2]}}$$

$$w_{\text{new}}^{[1]} = w_{\text{old}}^{[1]} - \eta (dw^{[1]})$$

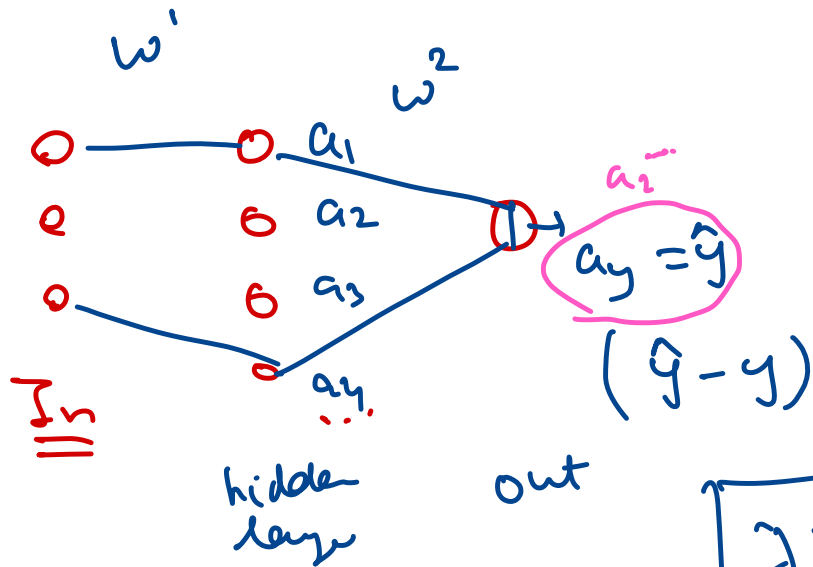
$\eta \Rightarrow$ learning rate

$$b_{\text{(new)}}^{[1]} = b_{\text{(old)}}^{[1]} - \eta (db^{[1]})$$

$$w_n^2 = w_0^2 - \eta dw^2$$

$$b_n^2 = b_0^2 - \eta db^2$$

$$L^i = - \left(y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right)$$



$$\frac{\partial J}{\partial w^1} = \left(\frac{\partial J}{\partial a^2} \cdot \frac{\partial a^2}{\partial z^2} \right) \cdot \frac{\partial z^2}{\partial a^1} \cdot \frac{\partial a^1}{\partial z^1} \cdot \frac{\partial z^1}{\partial w^1}$$

$$\frac{\partial J}{\partial w^2} = \frac{\partial J}{\partial a^2} \cdot \frac{\partial a^2}{\partial z^2} \cdot \frac{\partial z^2}{\partial w^2}$$

$$w^2 = w^1 - \eta \frac{\partial J}{\partial w^2}$$

$$b^1 = b^2 - \eta \frac{\partial J}{\partial b^1}$$

$$z^{[1]} = w^{[0]T} x + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = w^{[1]T} A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]})$$

(FP)

(BP)

$$\textcircled{1} dz^{[2]} = \frac{\partial J}{\partial z^{[2]}} = A^{[2]} - Y$$

$$\textcircled{2} dw^{[2]} = \frac{1}{n} dz^{[1]} \cdot A^{[1]T}$$

$$\textcircled{3} db^{[2]} = \frac{1}{n} \text{np.sum}(dz^{[2]}, \text{axis}=1, \text{keepdims}=\text{True})$$

output is
vector

ground truth

horz. summation

$$\textcircled{4} \quad dz^{[1]} = w^{[2]T} dz^{[2]} \quad \textcircled{*} \quad g^{[1]}(z^{[1]})$$

↓

elementwise operation

$$\textcircled{5} \quad dw^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

↪ input

$$\textcircled{6} \quad db^{[1]} = \frac{1}{m} \text{np.sum}(dz^{[1]}, \text{axis}=1, \text{keepdims}=\text{True})$$

Repeat till convergence

$$\left\{ \begin{array}{l} dw^{[2]} = \frac{\partial J}{\partial w^{[2]}} \\ db^{[2]} = \frac{\partial J}{\partial b^{[2]}} \end{array} \right.$$

$$dw^{[1]} = \frac{\partial J}{\partial w^{[1]}}$$

$$db^{[1]} = \frac{\partial J}{\partial b^{[1]}}$$

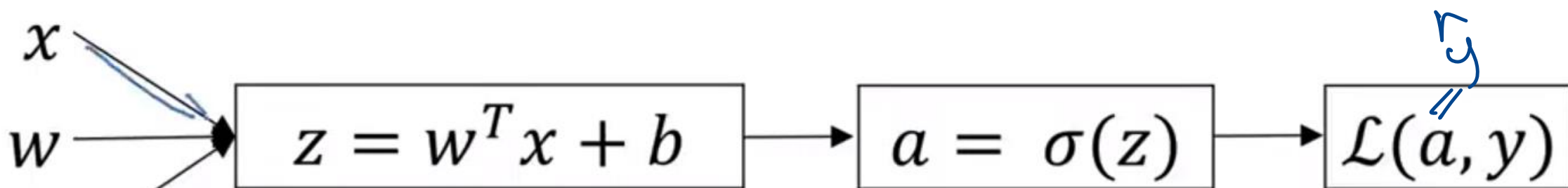
... and for other parameters.

$new \ W^{[1]} = old \ W^{[1]} - \eta \ dw^{[1]}$ *derivative w.r.t cost func.*

$new \ b^{[1]} = old \ b^{[1]} - \eta \ db^{[1]}$ *derivative w.r.t cost func.*

Logistic regression

$$(a = \hat{y})$$

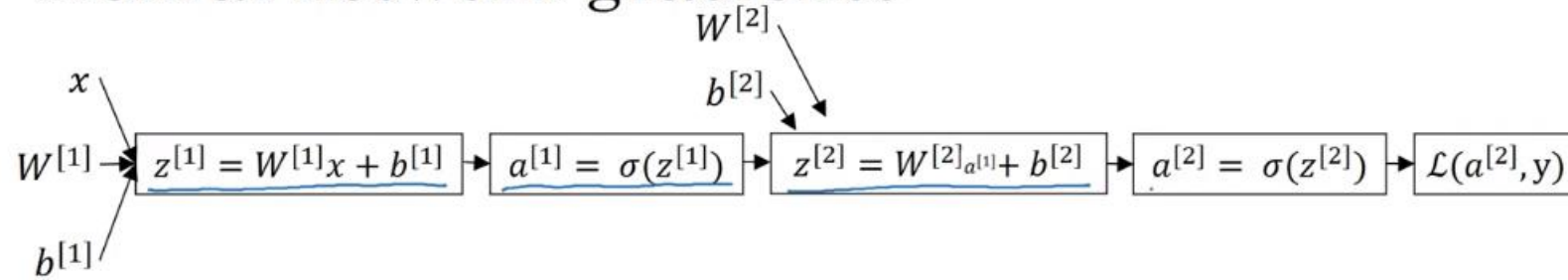


$$dz = a - y$$

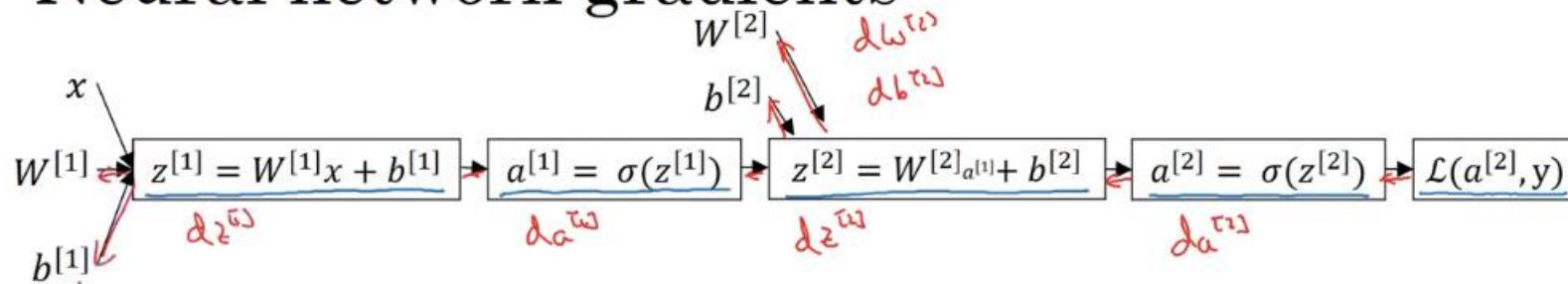
$$dz = da \cdot g'(z)$$
$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} g'(z)$$

$$da = \frac{\partial \mathcal{L}(a, y)}{\partial a}$$
$$= -\frac{y}{a} + \frac{1-y}{1-a}$$

Neural network gradients



Neural network gradients



Vectorization

$$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$\underset{(n^{[1]}, 1)}{dz^{[1]}} = W^{[2]T} dz^{[2]} * \underset{(n^{[1]}, m)}{g^{[1]}'(z^{[1]})}$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$\underline{dZ^{[2]}} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \text{np.sum}(dZ^{[2]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$\underset{(n^{[2]}, m)}{dZ^{[1]}} = \underset{(n^{[2]}, m)}{W^{[2]T} dZ^{[2]}} * \underset{(n^{[2]}, m)}{g^{[1]}'(Z^{[1]})}$$

↙ elementwise product

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} \text{np.sum}(dZ^{[1]}, \text{axis} = 1, \text{keepdims} = \text{True})$$

$$J(\cdot) = \frac{1}{m} \sum_{i=1}^n \mathcal{L}(\hat{y}_i, y_i)$$

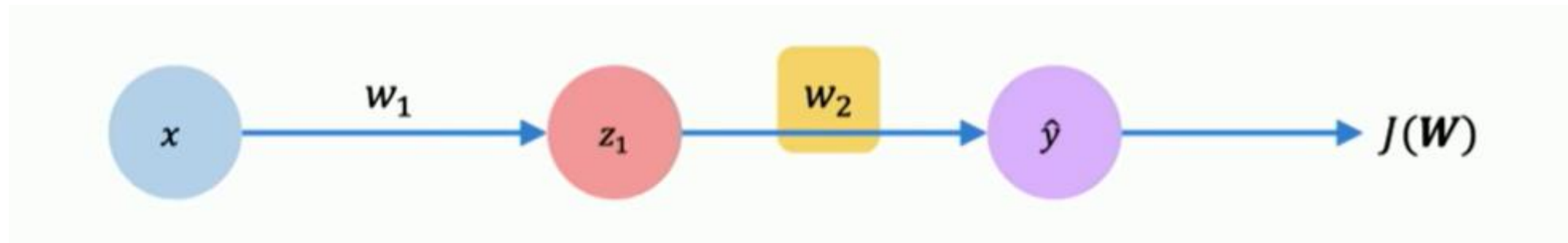
cu

The two assumptions we need about the cost function

- The goal of backpropagation is to compute the partial derivatives $\partial C / \partial w$ and $\partial C / \partial b$ of the cost function C with respect to any weight w or bias b in the network.
- For backpropagation to work, we need to make two main assumptions about the form of the cost function.
 1. The cost function can be written as an average $C = \frac{1}{n} \sum_x C_x$ over cost functions C_x for individual training examples, x .
 2. The cost function can be written as a function of the outputs from the neural network

Computing gradients using backpropagation

- How does a small change in one weight affect the final loss $J(W)$?



$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_2}$$

Computing gradients using backpropagation

$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \frac{\partial J(\mathbf{W})}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

A simple example using computation graphs

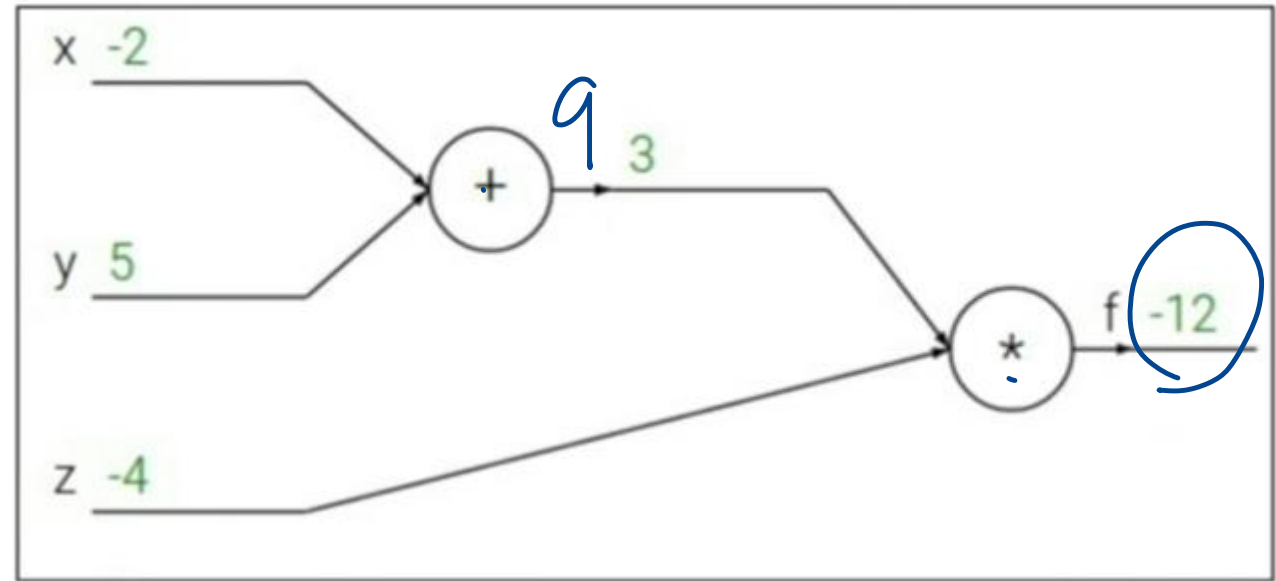
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



A simple example using computation graphs

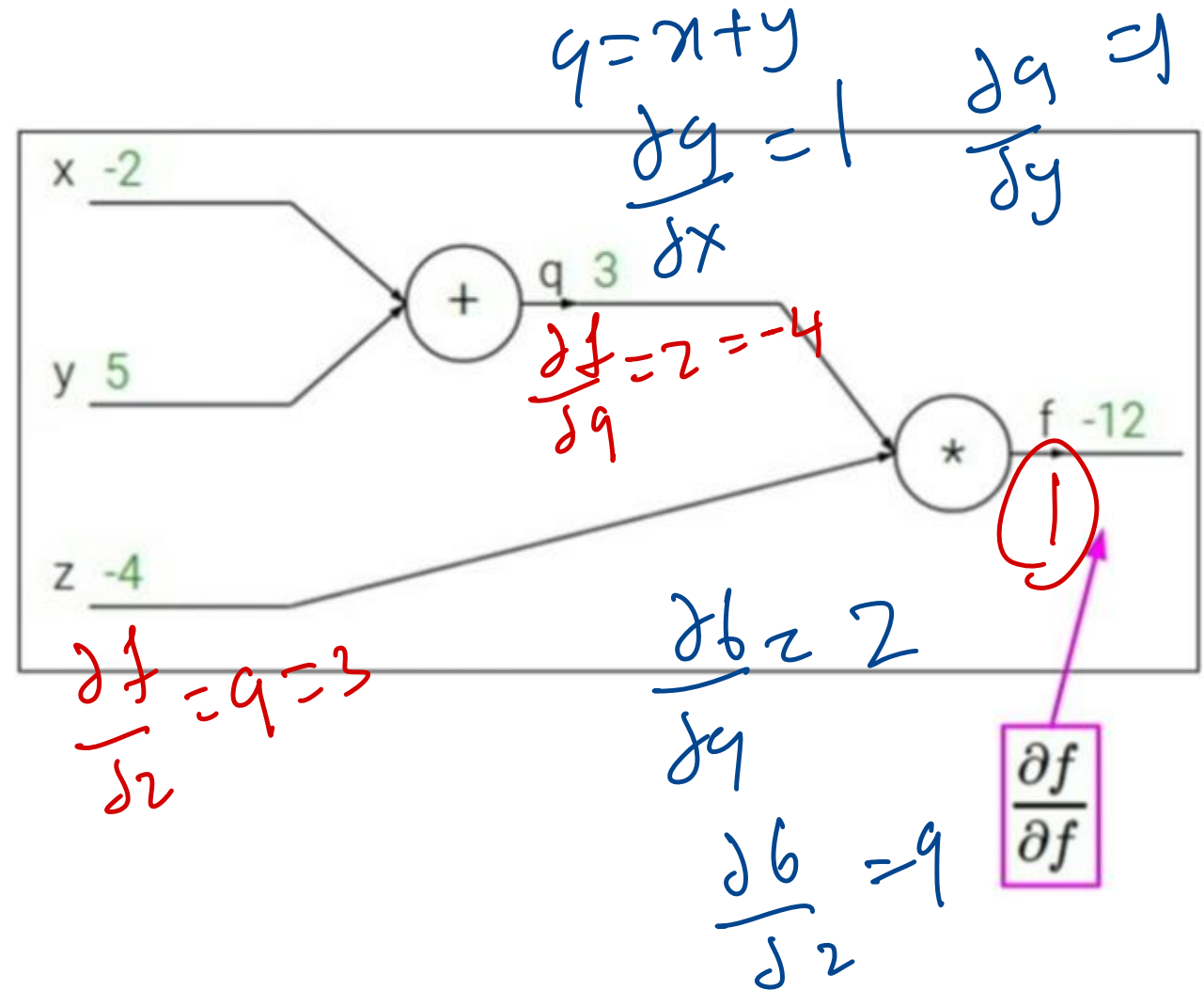
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



A simple example using computation graphs

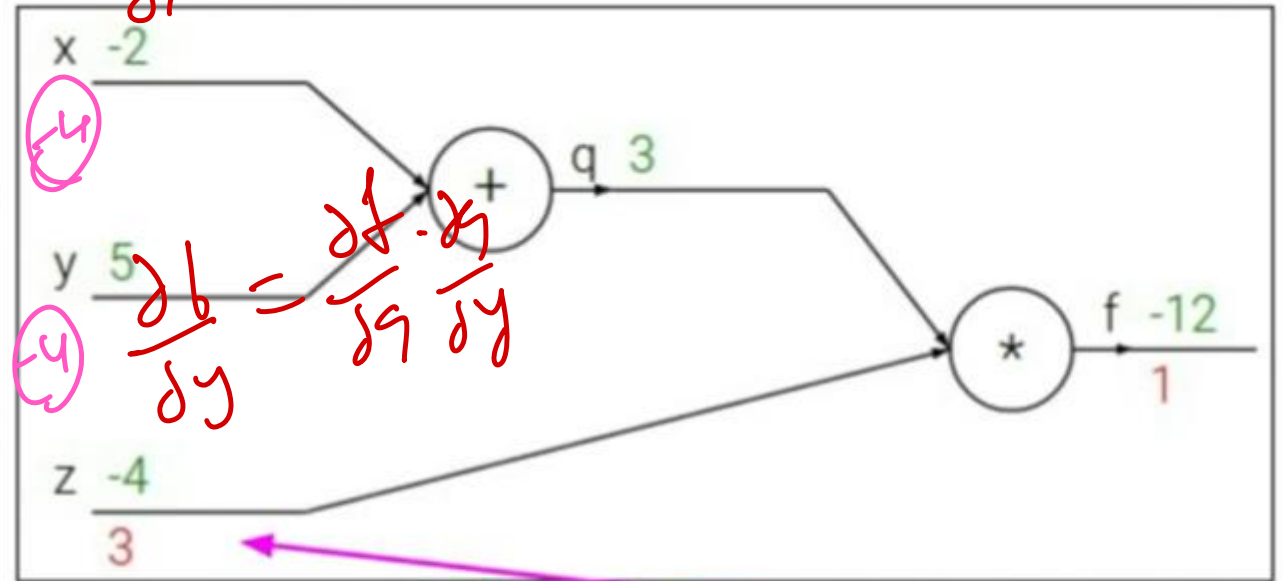
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

A simple example using computation graphs

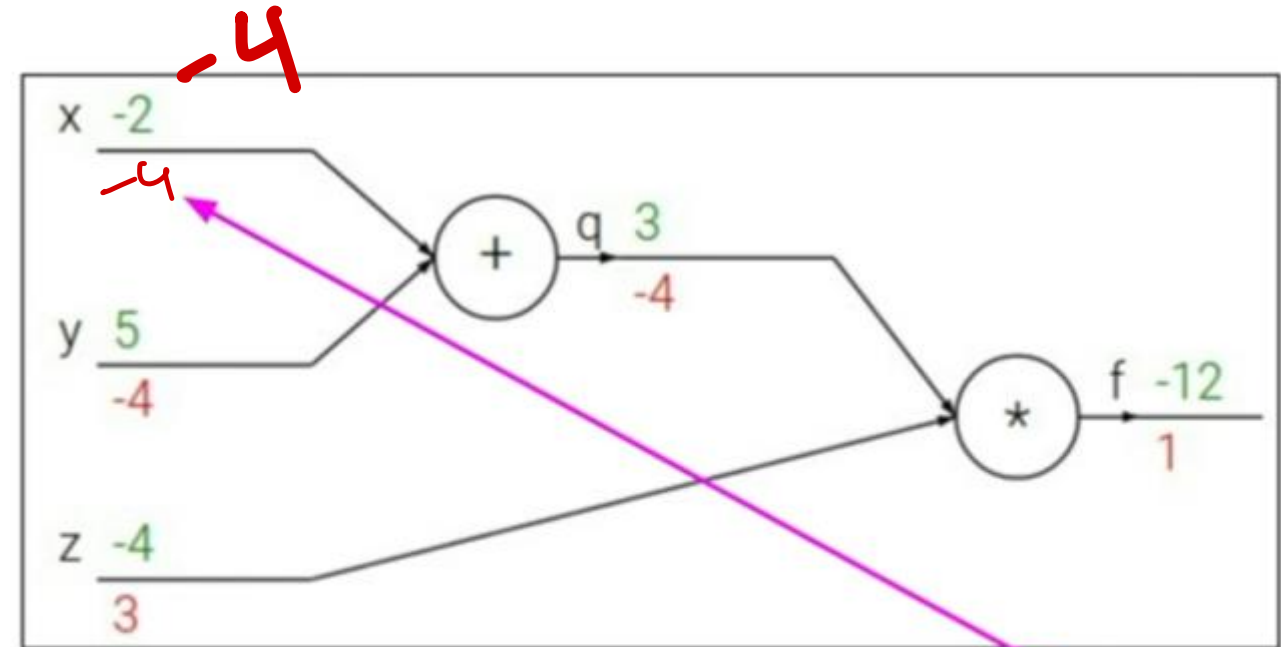
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

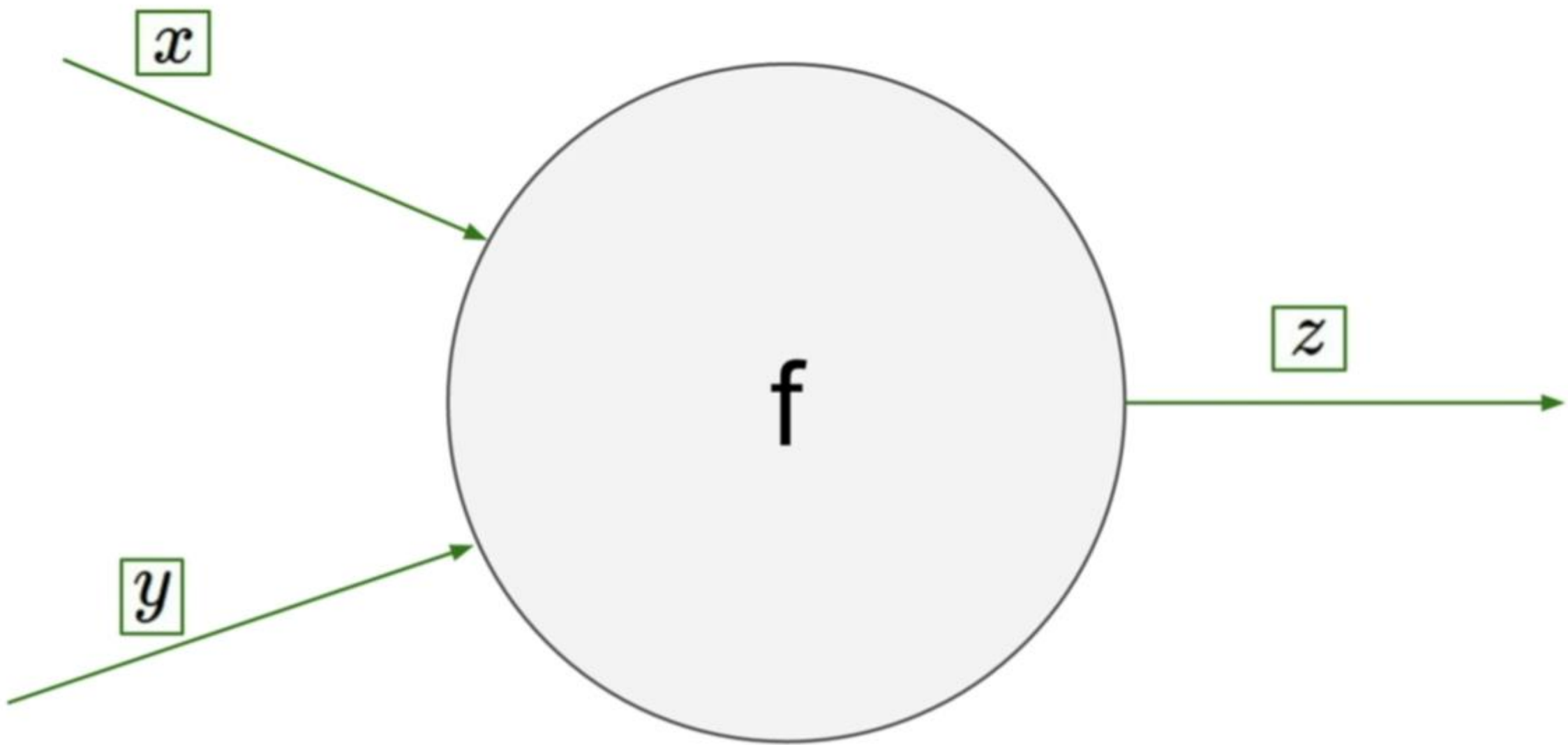
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

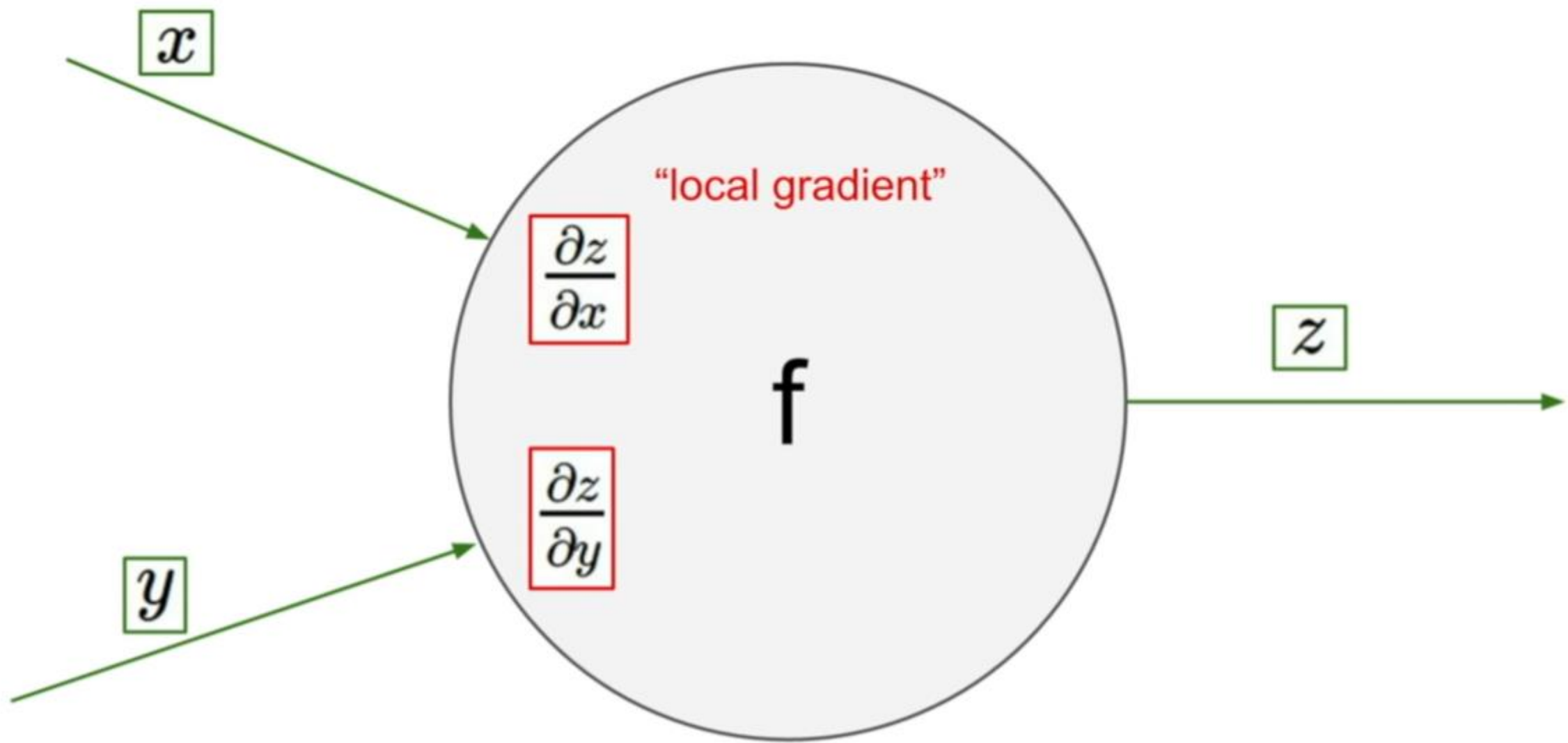


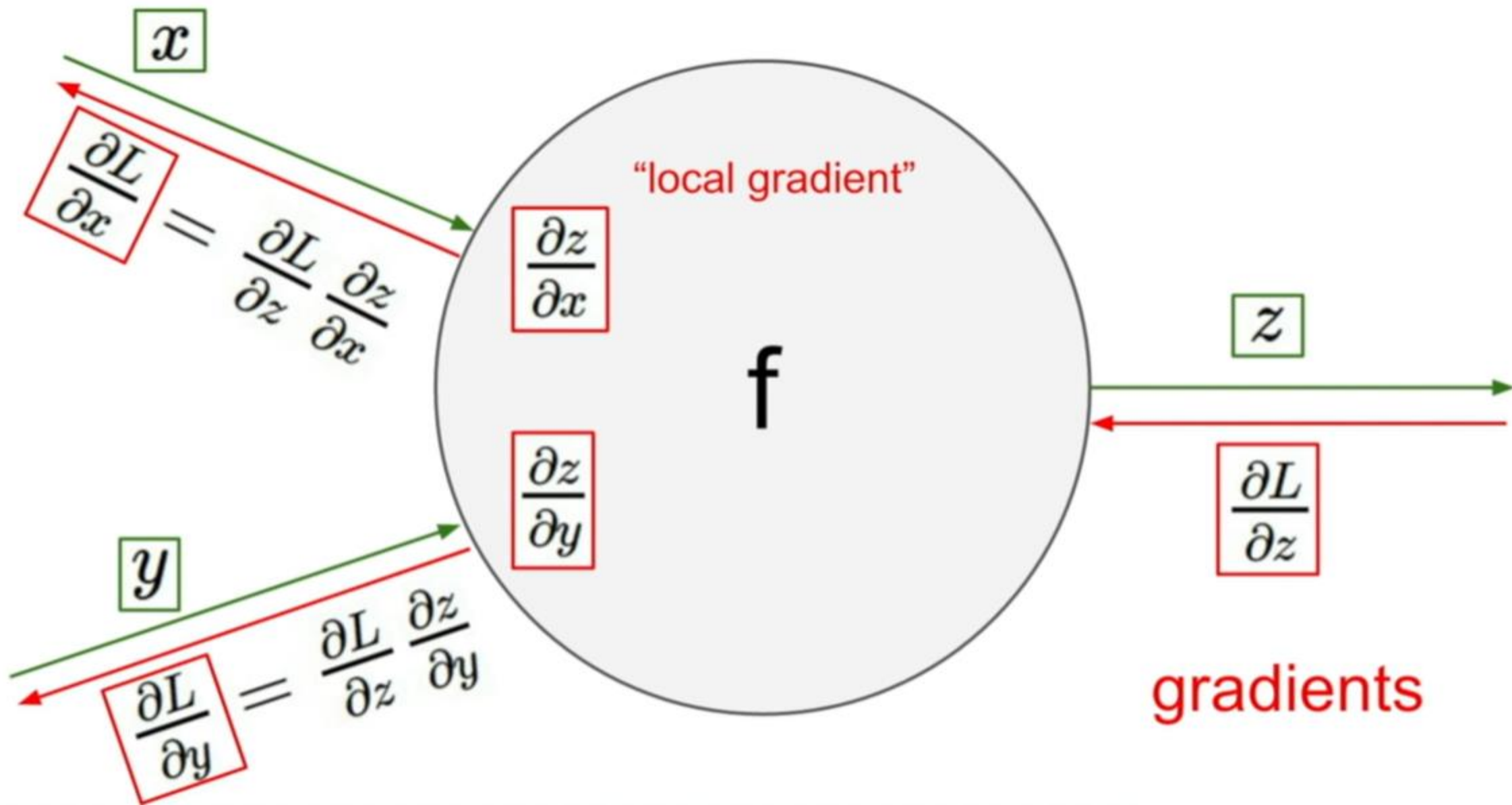
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

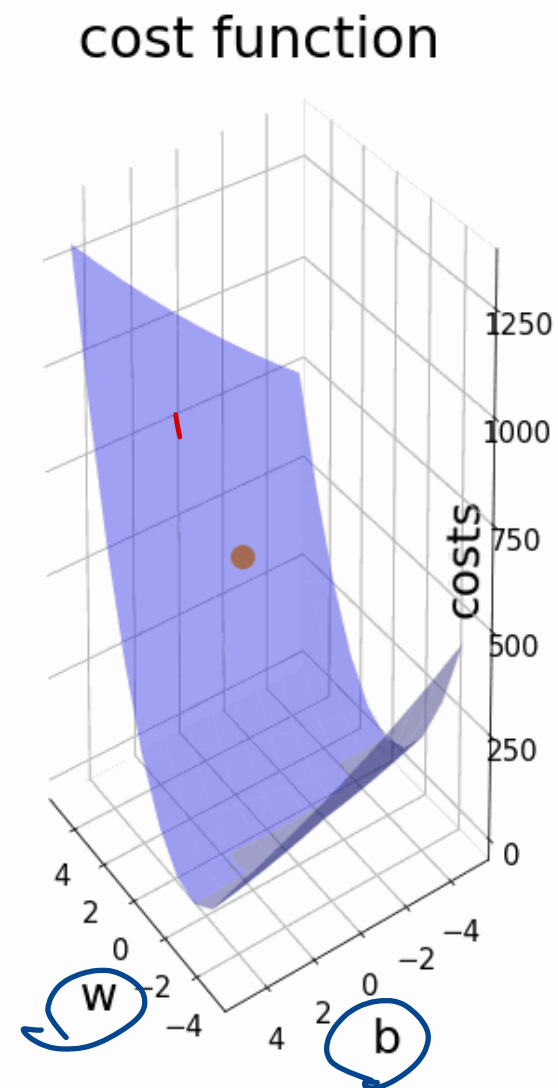
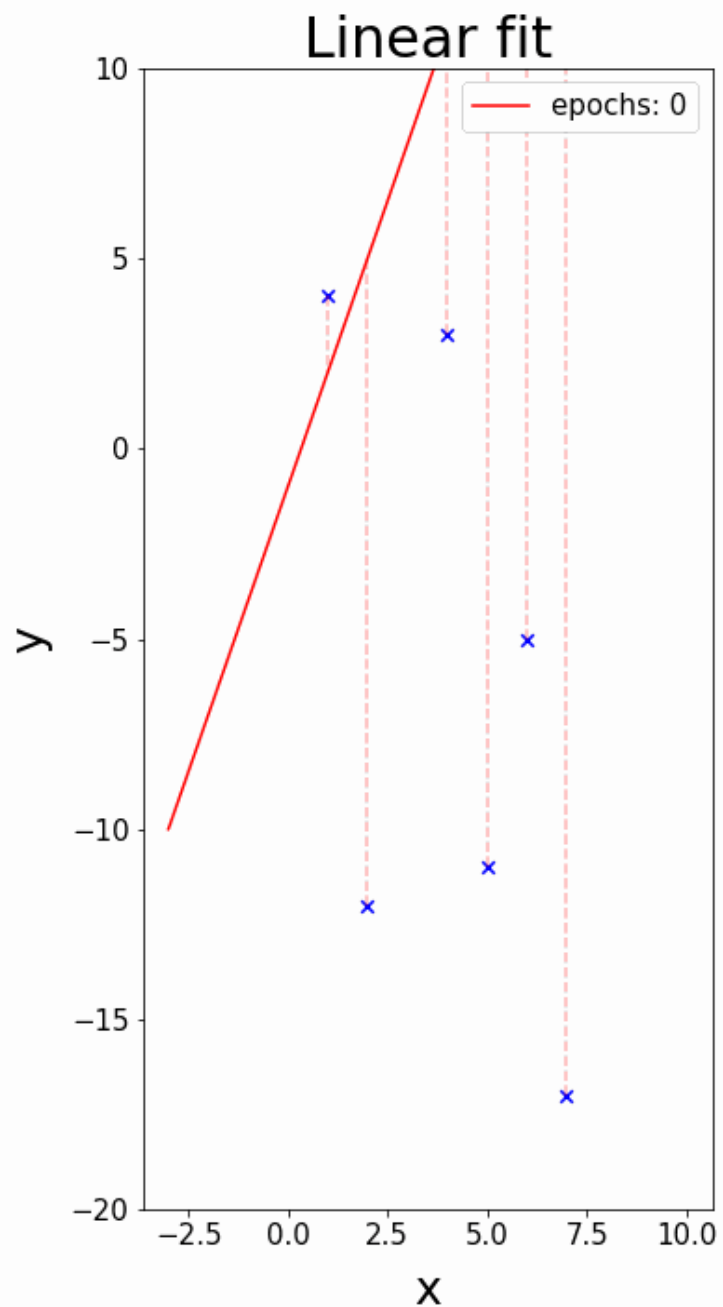
$$\frac{\partial f}{\partial x}$$



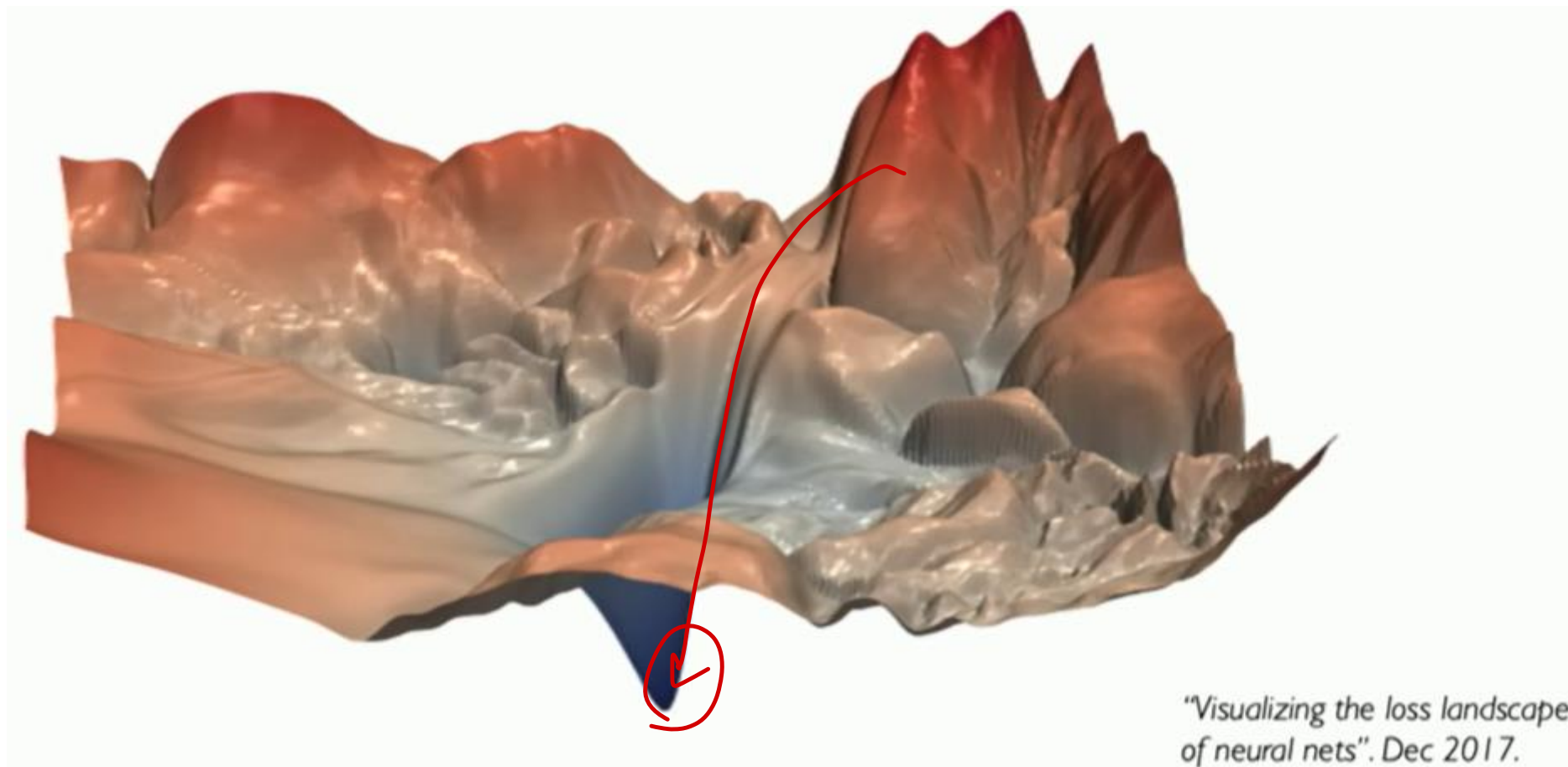




Visual example



Training Neural Networks is Difficult!



Loss functions can be difficult to optimize

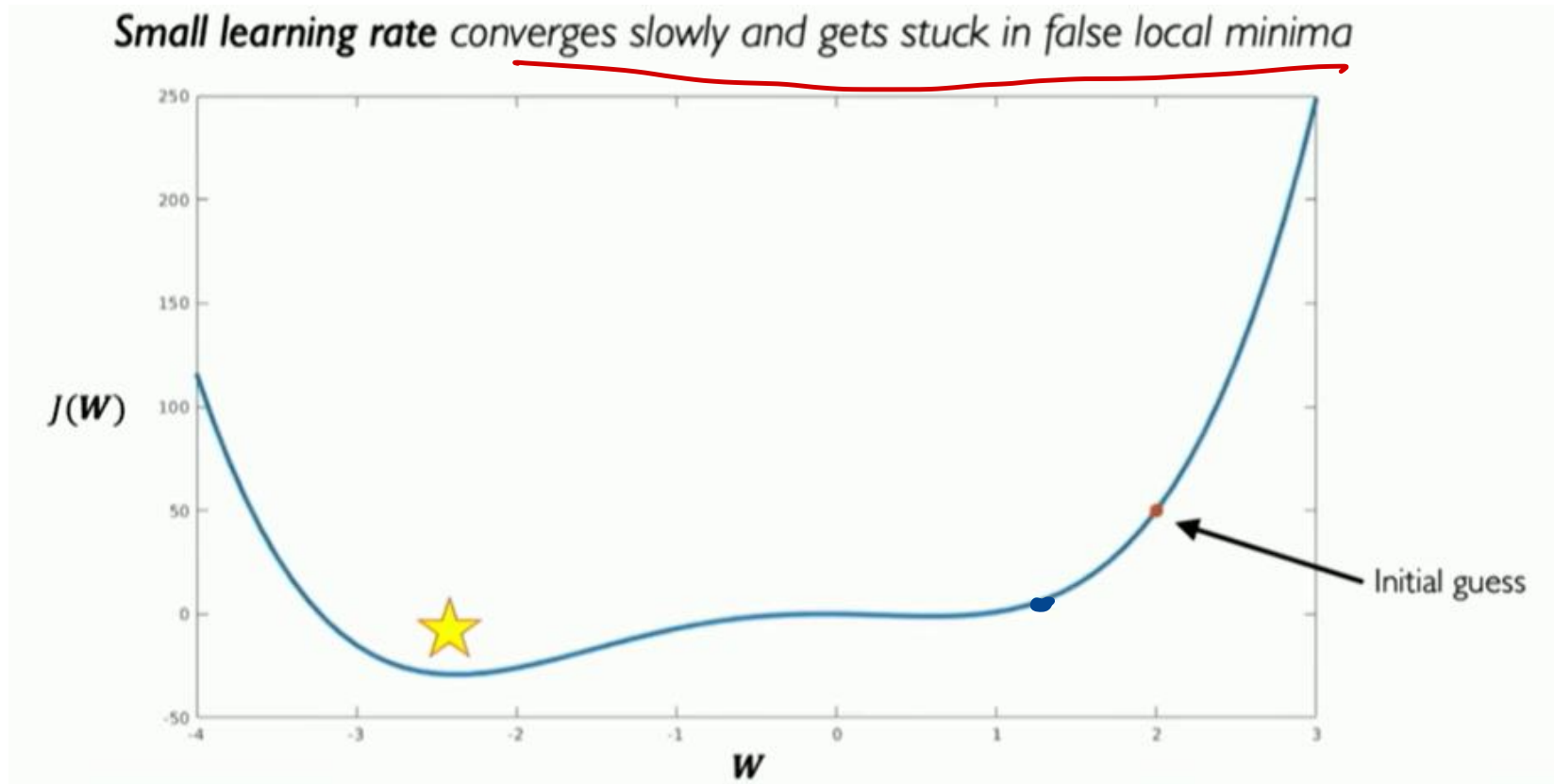
- Optimization through gradient descent:

Hyperparameter

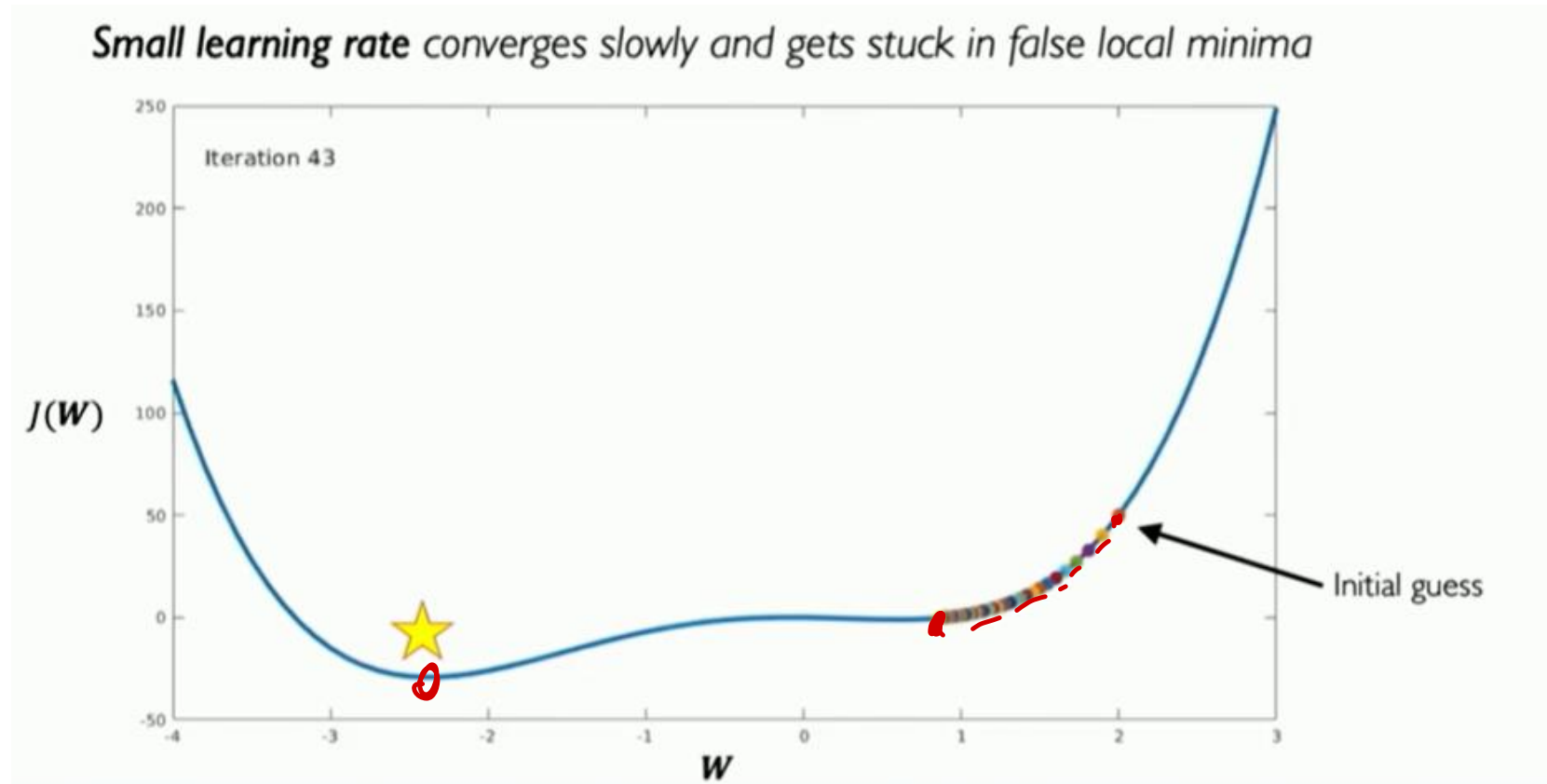
$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

How can we set the
learning rate?

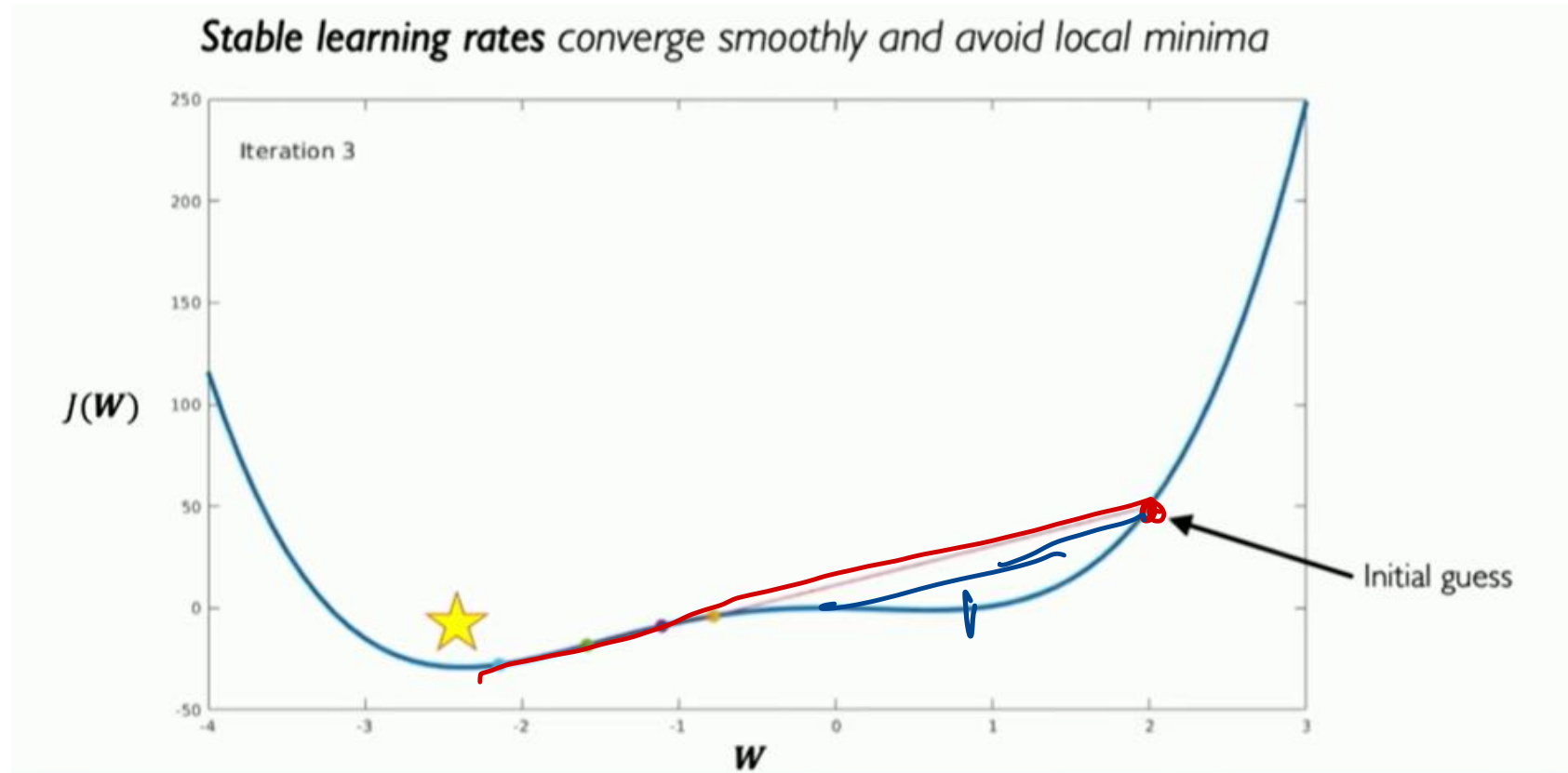
Setting the learning rate



Setting the learning rate



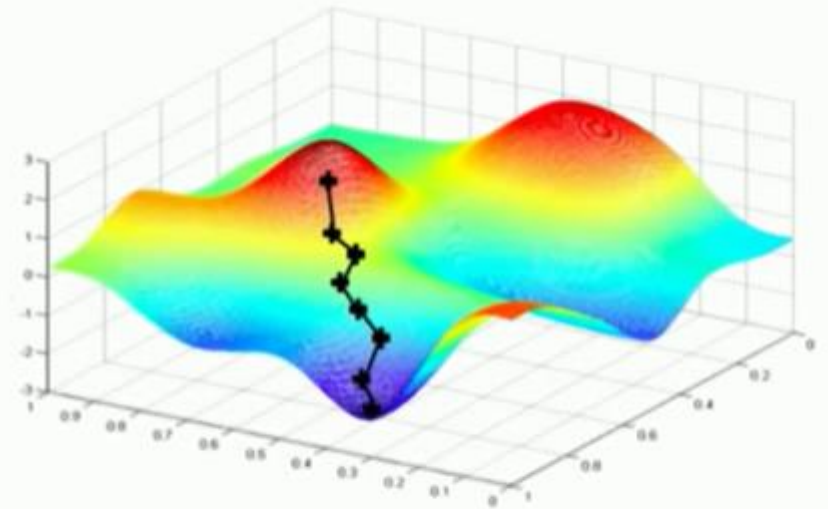
Setting the learning rate



Computing Gradient descent

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights



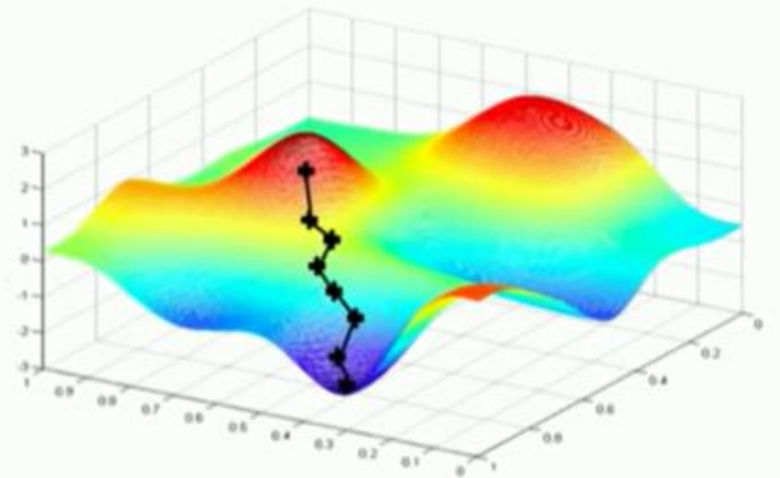
Can be very
computationally
intensive to compute!

Stochastic gradient descent

- Easy to compute but very noisy (stochastic)!

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick single data point i
4. Compute gradient, $\frac{\partial J_i(\mathbf{w})}{\partial \mathbf{w}}$
5. Update weights, $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}$
6. Return weights

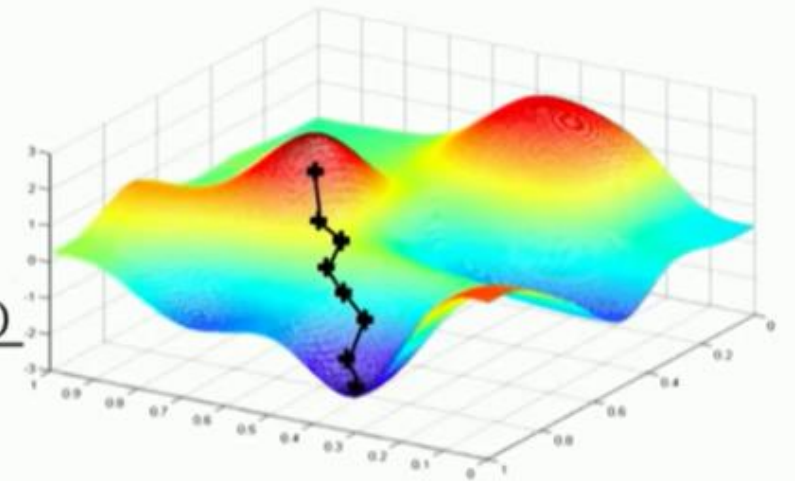


Mini-batch gradient descent

- Pick a mini-batch instead
- Fast to compute and a much better estimate of the true gradient

Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Pick batch of B data points
4. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(\mathbf{W})}{\partial \mathbf{W}}$
5. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
6. Return weights

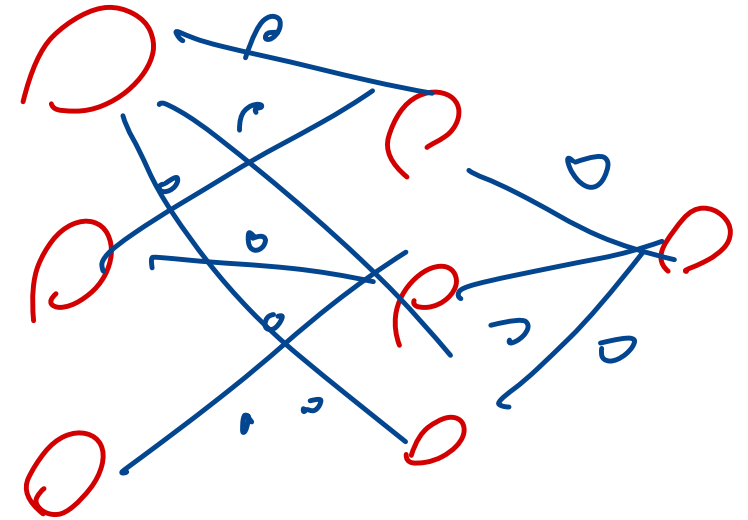


Gradient descent – types

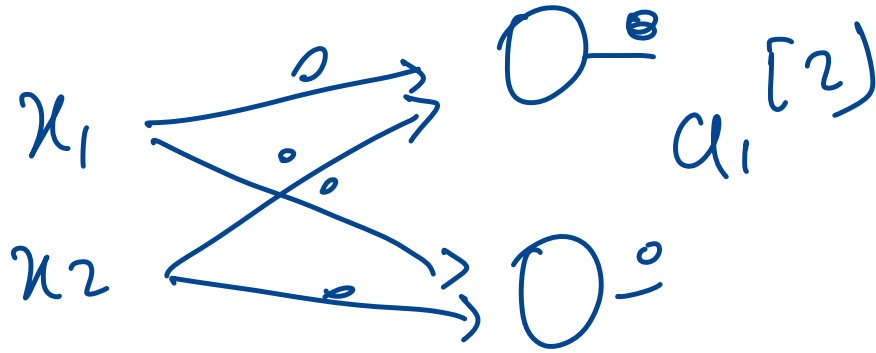
- **Batch Gradient Descent:** use the complete dataset available to compute the gradient of the cost function. Very slow because we need to calculate the gradient on the complete dataset to perform just one update, and if the dataset is large then it will be a difficult task.
- **Mini-batch Gradient Descent:** The dataset clustered into small groups of 'n' training datasets. In every iteration, we use a batch of 'n' training datasets to compute the gradient of the cost function. It is faster because it does not use the complete dataset.
- **Stochastic Gradient Descent:** The first step is to randomize the complete dataset. Then, we use only one training example in every iteration to calculate the gradient of the cost function for updating every parameter. It is faster for larger datasets also because it uses only one training example in each iteration.

Use of mini-batches while training

- More accurate estimation of gradient
- Smoother convergence
- Allows for larger learning rates
- Leads to faster training
 - Can parallelize computation
 - Achieve significant speed increase on GPUs



Initialize wts to zero!



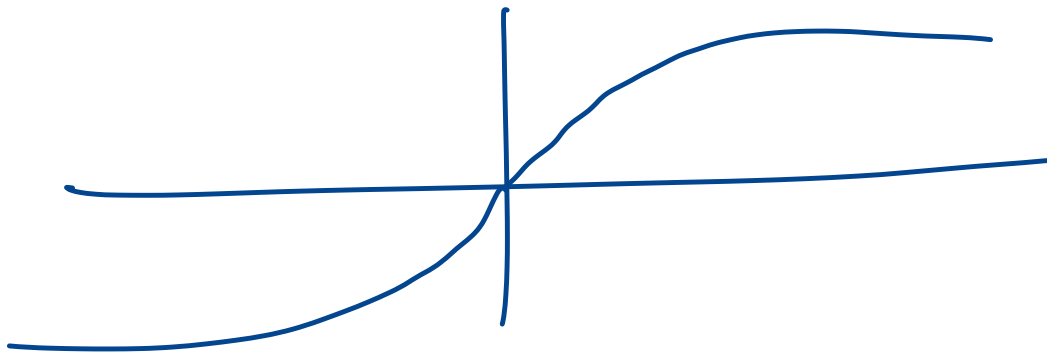
$$W^{[1]} = \text{np.random} \\ \text{ (2,2) } \times 0.01$$

$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$b^{[1]} = \text{np.zeros(2,1)}$$

if wts are initialised to zero they will NOT update.

OK



$w \uparrow \Rightarrow z \uparrow \Rightarrow a \approx -1$ or 1

slow down convergence

Keep w small. --

$0 \approx 1$

orders of mag.

- Content adapted from:
- MIT's Intro to Deep Learning. 6.S191. www.introtodeeplearning.com
- Stanford's CS 231n Deep Learning for Computer Vision <http://cs231n.stanford.edu/>
- Michael A. Nielsen, "Neural Networks and Deep Learning", Determination Press, 2015

CS229

0

0

0

 3×5 Input

0

5

0

0

0

0

(15)

(15)

3

0

0

0

(3)

1

0

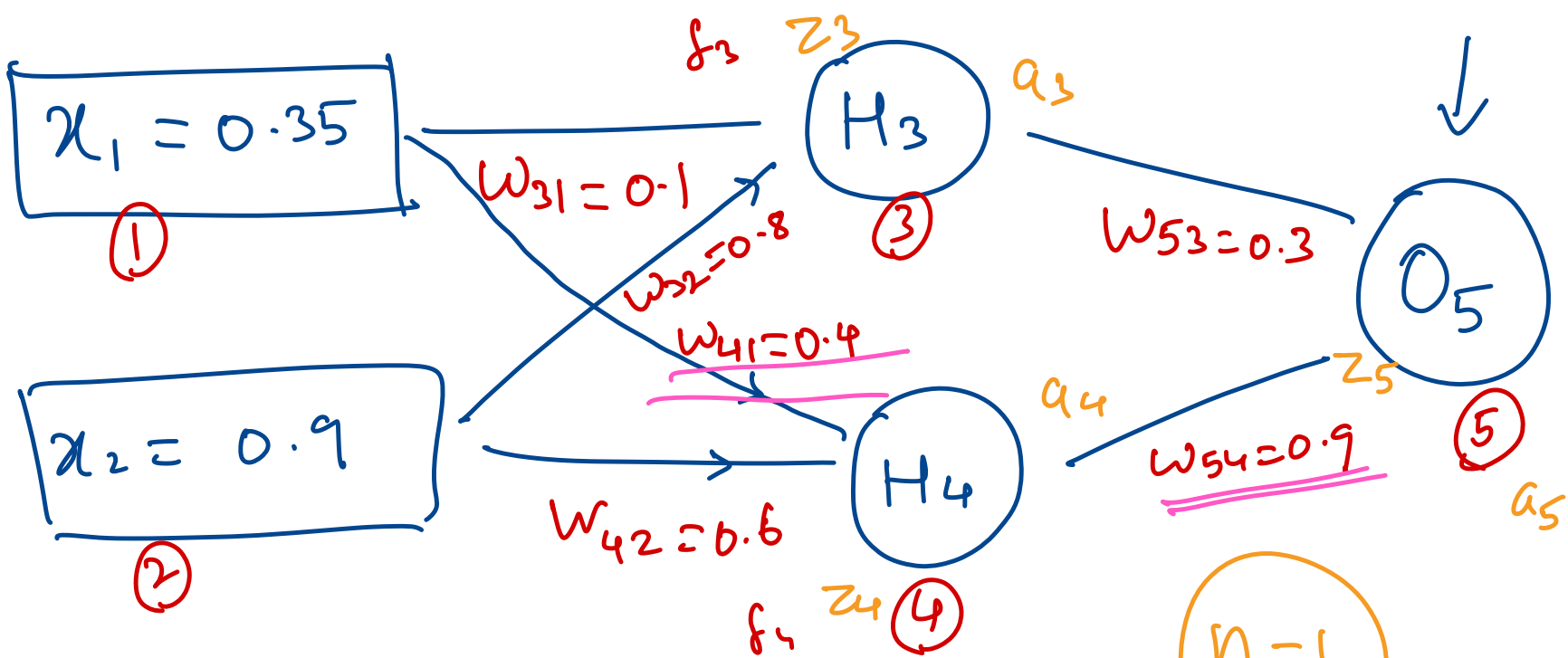
 6×1

49	15	16
37	45	34
42	48	
33	35	
39	30	
	26	
23	47	

How many
required ?

learning parameters are

$$3 \times 5 + 3 \times 5 + 3 \times 1 + 5 + 3 + 1 = \underline{\underline{42}}$$



$$z_3 = \sum (w_{ji} x_i)$$

$$= w_{31} x_1 + w_{32} x_2$$

$$= 0.1 \times 0.35 + 0.8 \times 0.9$$

$$z_3 = \underline{0.755}$$

$$z_4 = w_{41} x_1 + w_{42} x_2$$

$$= 0.4 \times 0.35 + 0.6 \times 0.9 = \underline{\underline{0.68}}$$

$$a_4 = \frac{1}{1 + e^{-z_4}} = 0.6637$$

$$a_3 = \frac{1}{1 + e^{-z_3}}$$

$$a_3 = \underline{0.68}$$

$$z_5 = w_{53}a_3 + w_{54}a_4 = \underline{0.801}$$

$$a_5 = \frac{1}{1 + e^{-0.801}} = \underline{\underline{0.69}}$$

$$y = 0.5$$

$$\zeta_{\text{error}} \Rightarrow y - a_5 = \textcircled{-0.19}$$

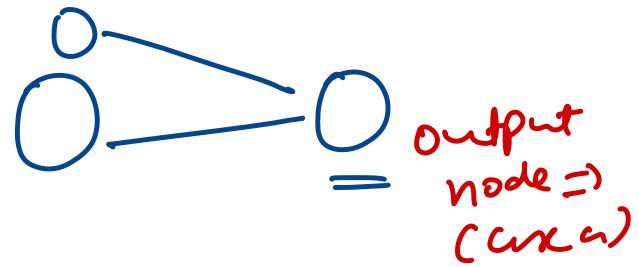
Back Propagation

$$\Delta w_{ji} = \eta \delta_j O_i$$

output $\leftarrow$ Δw_{ji} $\rightarrow$ input

$$\delta_j$$

$\rightarrow$ error measurement @ unit j



a) $\delta_j = O_j (1 - O_j) (t_j - O_j)$ $\Rightarrow$ j is output unit $\Rightarrow$ last layer!

t_j target value

b) $\delta_j = O_j (1 - O_j) \sum_k \delta_k w_{kj}$ if j is a hidden unit

$\sum_k \delta_k w_{kj}$ coming out of a particular hidden node!

$$\delta_5 = a_5(1-a_5)(y-a_5) \quad 0_j = a_5$$

$$= 0.69(1-0.69)(0.5-0.69)$$

$$= 0.69 \times 0.31 \times -0.19$$

$$= -0.0406$$

$$\delta_3 = a_3(1-a_3)w_{53} \delta_5$$

$$0.68(1-0.68)(0.3)(-0.0406)$$

$$= -0.00265$$

$$\delta_4 = a_4(1-a_4)w_{54} \cdot \delta_5$$

$$0.6637 \times (1-0.6637)(0.9) \cdot (-0.0406) = -0.0082$$

$$\Delta W_{54} = \eta \delta_5 a_4$$

$$\underline{\underline{\eta \delta_j U_i}}$$

$$= 1 \times -0.0406 \times 0.6637$$

$$= -0.0269$$

$$W_{54}^{\text{new}} = W_{54}^{\text{old}} + \Delta W_{54} = 0.9 - 0.0269$$

$$= \underline{\underline{0.8731}}$$

$$\Delta W_{41} = \eta \delta_4 x_1 = 1 \times -0.0082 \times 0.35$$

$$= -0.00287$$

$$W_{41}^{\text{new}} = W_{41}^{\text{old}} + \Delta W_{41} = 0.3971$$

Find updated $\Rightarrow W_{53}$

$\Rightarrow W_{31}$

$\Rightarrow W_{32}$

$\Rightarrow W_{42}$

Do 1 more forward propagation

i	j	w_{ji}	δ_j	x_i	η	updated w_{ji}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2721
4	5	0.9	-0.0406	0.6687	1	0.8731